# COLLEGE OF ENGINEERING TRIVANDRUM

**THIRUVANANTHAPURAM – 695016**




# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Fifth Semester**

## PCCSL507: NETWORKS LAB

Name : Mareo Manoj

KTU ID : TVE24CS099

## APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

**2026-2027**

# COLLEGE OF ENGINEERING TRIVANDRUM

**THIRUVANANTHAPURAM – 695016**




# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Fifth Semester**

## PCCSL507: NETWORKS LAB

Certified that this is a bonafide record of work done by Mareo Manoj (TVE24CS099) of S5 CSE (Batch-2) class in the Networks Lab during the year 2026-2027.

External Examiner

Faculty Member in charge

# Experiment No. 1

# FAMILIARIZATION OF LINUX NETWORKING COMMANDS

---

## AIM

Familiarize Linux networking commands - ifconfig, ifplugstatus, iftop, ping, ip, traceroute, mtr, netstat, whois, nmap, nmcli, speedtest-cli, bmon, nslookup, tcpdump

## COMMANDS

### 1. ifconfig (Interface Configuration)

Used to view and configure network interfaces. It displays the IP address, subnet mask, MAC address and status of each interface. It has largely been replaced by the `ip` command in modern distributions, but is still widely used for basic troubleshooting.

```
ifconfig -a    # all interfaces, including inactive
ones
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ ifconfig -a
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 1605  bytes 158702 (154.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1605  bytes 158702 (154.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlp1s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.17  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::97d0:a274:dbb4:b89a  prefixlen 64  scopeid 0x20<link>
        ether 66:c1:07:7e:39:18  txqueuelen 1000  (Ethernet)
        RX packets 305998  bytes 404599320 (385.8 MiB)
        RX errors 0  dropped 1  overruns 0  frame 0
        TX packets 140111  bytes 28970804 (27.6 MiB)
        TX errors 0  dropped 7 overruns 0  carrier 0  collisions 0

student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 1: Terminal Output - `ifconfig`

### 2. ifplugstatus

Checks whether a network cable is physically connected to an Ethernet interface (link detected or not). It is part of the `ifplugd` package.

```
ifplugstatus [interface]
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ ifplugstatus
lo: link beat detected
wlan0: link beat detected
eth0: unplugged
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 2: Terminal Output - `ifplugstatus`

### 3. iftop

A real-time bandwidth monitoring tool that shows which hosts are sending and receiving data through an interface, and how much bandwidth each connection uses.

```
iftop [options]
```

| Option | Description |
|---|---|
| `-i interface` | Monitor a specific network interface. |
| `-n` | Do not resolve hostnames; display IP addresses only. |
| `-N` | Display port numbers instead of service names. |
| `-P` | Show both port numbers and host information. |
| `-B` | Display bandwidth in bytes instead of bits. |
| `-h` | Display help information. |

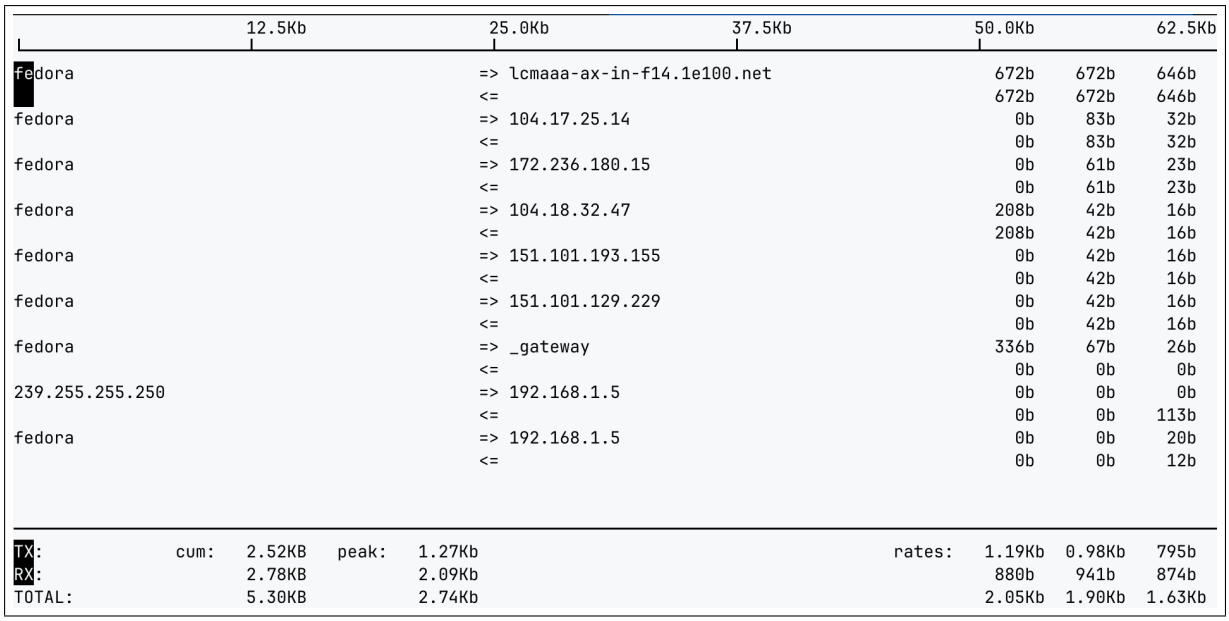



Figure 3: Terminal Output - `iftop`

## 4. ping

Checks whether a remote host is reachable and measures the Round-Trip Time (RTT) of packets between the local system and the destination.

```
ping [options] hostname/IP_address
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ ping google.com
PING google.com (192.178.173.139) 56(84) bytes of data.
64 bytes from lcbome-in-f139.1e100.net (192.178.173.139): icmp_seq=1 ttl=111 time=242 ms
64 bytes from lcbome-in-f139.1e100.net (192.178.173.139): icmp_seq=2 ttl=111 time=73.3 ms
64 bytes from lcbome-in-f139.1e100.net (192.178.173.139): icmp_seq=3 ttl=111 time=186 ms
64 bytes from lcbome-in-f139.1e100.net (192.178.173.139): icmp_seq=4 ttl=111 time=108 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 73.308/152.308/242.117/66.060 ms
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 4: Terminal Output - `ping`

## 5. ip

A powerful utility to display and configure interfaces, IP addresses, routing tables and network settings. It is the modern replacement for `ifconfig`, `route` and `netstat`.

```
ip [options] object command
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: wlp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 66:c1:07:7e:39:18 brd ff:ff:ff:ff:ff:ff permaddr 00:41:0e:46:f3:e3
    altname wlx00410e46f3e3
    inet 192.168.1.17/24 brd 192.168.1.255 scope global dynamic noprefixroute wlp1s0
       valid_lft 83292sec preferred_lft 83292sec
    inet6 fe80::97d0:a274:dbb4:b89a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 5: Terminal Output - `ip`

### 6. traceroute

Traces the path taken by packets from the source to a destination host, listing every intermediate router (hop). It helps identify delays and connectivity problems.

```
traceroute hostname/IP_address
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ traceroute google.com
traceroute to google.com (142.250.182.78), 30 hops max, 60 byte packets
 1  _gateway (192.168.1.1)  2.063 ms  1.968 ms  2.089 ms
 2  172.25.0.21 (172.25.0.21)  15.536 ms  15.944 ms  15.922 ms
 3  185.252.88.202.asianet.co.in (202.88.252.185)  15.635 ms * *
 4  45.243.88.202.asianet.co.in (202.88.243.45)  15.824 ms  15.801 ms  15.778 ms
 5  * * *
 6  130.230.88.202.asianet.co.in (202.88.230.130)  36.344 ms  38.205 ms  41.232 ms
 7  21.252.88.202.asianet.co.in (202.88.252.21)  44.862 ms  32.580 ms  35.371 ms
 8  lcmaaa-ax-in-f14.1e100.net (142.250.182.78)  39.082 ms  42.577 ms  46.056 ms
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 6: Terminal Output - `traceroute`

### 7. mtr (My Traceroute)

Combines the functionality of `ping` and `traceroute`. It continuously sends packets and shows real-time latency and packet-loss statistics for each hop.

```
mtr hostname/IP_address
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ mtr -rwc 10 google.com
Start: 2026-10-20T12:32:58+0530
HOST: fedora                          Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- _gateway                       0.0%    10  127.6  49.3   1.5 237.4  79.6
  2.|-- 172.25.0.21                    0.0%    10    8.2  49.2   3.2 119.2  43.1
  3.|-- 185.252.88.202.asianet.co.in 20.0%    10   10.0  49.2   4.3 127.5  52.1
  4.|-- 45.243.88.202.asianet.co.in  90.0%    10   66.4  66.4  66.4  66.4   0.0
  5.|-- 57.243.88.202.asianet.co.in  80.0%    10  228.9 165.3 101.7 228.9  90.0
  6.|-- 130.230.88.202.asianet.co.in  0.0%    10   42.3 103.8  35.2 181.5  44.4
  7.|-- 21.252.88.202.asianet.co.in   0.0%    10  224.3 107.5  22.5 224.3  65.1
  8.|-- maa05s20-in-f14.1e100.net     0.0%    10  105.7 111.7  23.1 258.8  78.5
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 7: Terminal Output - `mtr`

### 8. netstat (Network Statistics)

Displays network connections, routing tables, interface statistics and listening ports.

```
netstat [options]
```

| Option | Description |
|---|---|
| `-t` | Display TCP connections. |
| `-u` | Display UDP connections. |
| `-l` | Display listening ports only. |
| `-n` | Show numerical addresses and port numbers. |
| `-r` | Display the routing table. |
| `-i` | Display network interface statistics. |
| `-a` | Show all active and listening connections. |

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:5355            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN
tcp6       0      0 ::1:631                 :::*                    LISTEN
tcp6       0      0 :::39000                :::*                    LISTEN
tcp6       0      0 :::22                   :::*                    LISTEN
tcp6       0      0 :::5355                 :::*                    LISTEN
tcp6       0      0 :::38000                :::*                    LISTEN
udp        0      0 0.0.0.0:47734           0.0.0.0:*
udp        0      0 192.168.1.17:3702       0.0.0.0:*
udp        0      0 239.255.255.250:3702    0.0.0.0:*
udp        0      0 127.0.0.54:53           0.0.0.0:*
udp        0      0 127.0.0.53:53           0.0.0.0:*
udp        0      0 127.0.0.1:323           0.0.0.0:*
udp        0      0 224.0.0.251:5353        0.0.0.0:*
udp        0      0 0.0.0.0:5353            0.0.0.0:*
udp        0      0 0.0.0.0:5355            0.0.0.0:*
udp6       0      0 :::47040                :::*
udp6       0      0 fe80::97d0:a274:db:3702 :::*
udp6       0      0 ff02::c:3702            :::*
udp6       0      0 ::1:323                 :::*
```

Figure 8: Terminal Output - `netstat`

### 9. whois

Retrieves information about a domain name, IP address or network owner from WHOIS databases, such as registrar details, registration dates and contact information.

```
whois domain_name
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ whois google.com | head -30
   Domain Name: GOOGLE.COM
   Registry Domain ID: 2138514_DOMAIN_COM-VRSN
   Registrar WHOIS Server: whois.markmonitor.com
   Registrar URL: http://www.markmonitor.com
   Updated Date: 2019-09-09T15:39:04Z
   Creation Date: 1997-09-15T04:00:00Z
   Registry Expiry Date: 2028-09-14T04:00:00Z
   Registrar: MarkMonitor Inc.
   Registrar IANA ID: 292
   Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
   Registrar Abuse Contact Phone: +1.2086851750
   Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
   Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
   Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
   Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
   Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
   Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
   Name Server: NS1.GOOGLE.COM
   Name Server: NS2.GOOGLE.COM
   Name Server: NS3.GOOGLE.COM
   Name Server: NS4.GOOGLE.COM
   DNSSEC: unsigned
```

Figure 9: Terminal Output - `whois`

### 10. nmap (Network Mapper)

An open-source network scanning tool used for network discovery and security auditing: host discovery, port scanning, service identification and operating system detection.

```
nmap [options] target
```

| Option | Description |
|---|---|
| `-sS` | TCP SYN (stealth) scan. |
| `-sV` | Detect service versions. |
| `-O` | Detect the operating system. |
| `-A` | Aggressive scan (OS detection, version detection, traceroute). |
| `-p` | Scan specific port(s). |
| `-F` | Fast scan of common ports. |
| `-v` | Display verbose output. |

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ nmap -sV localhost
Starting Nmap 7.92 ( https://nmap.org ) at 2026-10-20 12:36 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00015s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (conn-refused)
PORT    STATE SERVICE VERSION
22/tcp  open  ssh     OpenSSH 10.2 (protocol 2.0)
631/tcp open  ipp     CUPS 2.4

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.49 seconds
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 10: Terminal Output - `nmap`

### 11. nmcli (NetworkManager Command Line Interface)

Manages network connections on Linux systems that use NetworkManager. It can show network status, configure IP addresses, connect to Wi-Fi, enable or disable interfaces and help troubleshoot connections.

```
nmcli [options] object command
```

| Object | Command | Description |
|---|---|---|
| `device` | `status` | Displays the status of network devices. |
| `connection` | `show` | Lists all configured network connections. |
| `connection` | `up` | Activates a network connection. |
| `connection` | `down` | Deactivates a network connection. |
| `radio` | `wifi on` | Enables Wi-Fi. |
| `radio` | `wifi off` | Disables Wi-Fi. |

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ nmcli device status
DEVICE  TYPE      STATE                   CONNECTION
wlp1s0  wifi      connected               Campus-WiFi
lo      loopback  connected (externally)  lo
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 11: Terminal Output - `nmcli`

### 12. speedtest-cli

Measures the performance of an Internet connection: download speed, upload speed and ping (latency).

```
speedtest-cli [options]
```

| Option | Description |
|---|---|
| `--simple` | Display simplified output (ping, download, upload). |
| `--list` | List available Speedtest servers. |
| `--server ID` | Use a specific server for testing. |
| `--share` | Generate and share the test result as an image link. |
| `--version` | Display the installed version. |
| `--help` | Display help information. |

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ speedtest-cli --simple
Ping: 427.72 ms
Download: 2.28 Mbit/s
Upload: 3.60 Mbit/s
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 12: Terminal Output - `speedtest-cli`

### 13. bmon (Bandwidth Monitor)

Monitors bandwidth usage in real time, showing incoming (RX) and outgoing (TX) traffic statistics for each interface along with a graph of activity over time.

```
bmon [options]
```

| Option | Description |
|---|---|
| `-p interface` | Monitor a specific network interface. |
| `-a` | Display all network interfaces. |
| `-o format` | Specify the output format. |
| `-h` | Display help information. |
| `-v` | Display version information. |

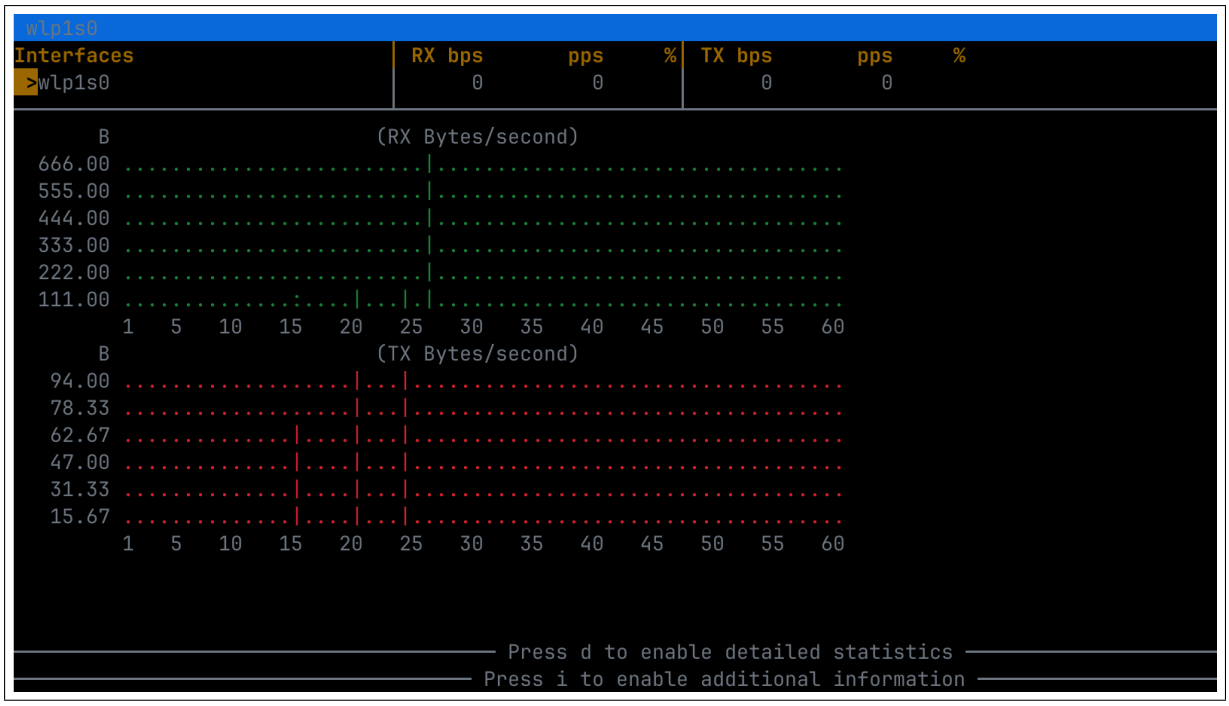



Figure 13: Terminal Output - `bmon`

### 14. nslookup (Name Server Lookup)

Queries the Domain Name System (DNS) to obtain the IP address, mail servers and name servers associated with a domain name. It is used to verify DNS records and troubleshoot DNS issues.

```
nslookup domain_name
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ nslookup google.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.182.78
;; communications error to 127.0.0.53#53: timed out
Name:   google.com
Address: 2404:6800:4007:810::200e

student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 14: Terminal Output - `nslookup`

### 15. tcpdump

A command-line packet analyzer used to capture and inspect network packets in real time, for troubleshooting, protocol analysis and security monitoring.

```
tcpdump [options]
```

```
student@cse-lab:~/Networks-Lab-Sem-5-CS$ sudo tcpdump -i wlp1s0 -c 10
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlp1s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:41:58.129976 ARP, Request who-has fedora tell _gateway, length 28
12:41:58.130001 ARP, Reply fedora is-at 66:c1:07:7e:39:18 (oui Unknown), length 28
12:41:58.217805 ARP, Request who-has _gateway tell fedora, length 28
12:41:58.225211 ARP, Reply _gateway is-at bc:62:d2:7b:2b:18 (oui Unknown), length 28
12:42:04.273776 IP fedora.38325 > _gateway.domain: 35307+ [1au] A? fedoraproject.org. (46)
12:42:04.293613 IP _gateway.domain > fedora.38325: 35307 6/0/1 A 13.250.125.204, A 38.145.32.21, A 38.145.32.20, A 3.34
.177.3, A 52.221.221.142, A 3.110.169.64 (142)
12:42:04.295086 IP fedora.40244 > ec2-13-250-125-204.ap-southeast-1.compute.amazonaws.com.http: Flags [S], seq 25810754
77, win 64240, options [mss 1460,sackOK,TS val 621461736 ecr 0,nop,wscale 10], length 0
12:42:04.348293 IP ec2-13-250-125-204.ap-southeast-1.compute.amazonaws.com.http > fedora.40244: Flags [S.], seq 3009059
823, ack 2581075478, win 62643, options [mss 1452,sackOK,TS val 3525416950 ecr 621461736,nop,wscale 10], length 0
12:42:04.348369 IP fedora.40244 > ec2-13-250-125-204.ap-southeast-1.compute.amazonaws.com.http: Flags [.], ack 1, win 6
3, options [nop,nop,TS val 621461789 ecr 3525416950], length 0
12:42:04.348601 IP fedora.40244 > ec2-13-250-125-204.ap-southeast-1.compute.amazonaws.com.http: Flags [P.], seq 1:94, a
ck 1, win 63, options [nop,nop,TS val 621461789 ecr 3525416950], length 93: HTTP: GET /static/hotspot.txt HTTP/1.1
10 packets captured
18 packets received by filter
0 packets dropped by kernel
student@cse-lab:~/Networks-Lab-Sem-5-CS$
```

Figure 15: Terminal Output - `tcpdump`

### RESULT

The Linux networking commands were executed successfully, and network configuration, connectivity, traffic, and performance information were observed and analyzed.

# Experiment No. 2

# WIRESHARK HTTP PACKET ANALYSIS

---

## AIM

Start your web browser and clear the browser's cache memory. Open Wireshark and start capturing. Then visit any webpage of your choice. Type `http` in the filter field of the Wireshark and click Apply so that only HTTP messages are displayed. After enough packets have been captured, select the Capture from the pull-down menu and select Stop to stop capturing. Using the captured information, determine the following:

(a) The source IP address and destination IP address of the first GET message.

(b) The medium format, the language, the encoding, and the character set that the client can accept (use the first GET message).

(c) The URL of the website and the user agent (use the first GET message).

(d) The source IP address and destination IP address of the first response message.

(e) The status codes for the first response message.

(f) When the HTML file that you are retrieving was last modified at the server.

(g) Value of the content-length field of the first response message.

(h) How long it took from the time the GET message was sent until the response message was received (use the timestamps of a GET message and that of the corresponding response message; by default, the time column's value is the amount of time in seconds since Wireshark tracing began).

(i) The HTTP version of your browser.

## PROCEDURE

1. Open the web browser and clear the browser cache.
2. Open Wireshark and select the active network interface.
3. Click **Start Capturing Packets**.

4. Open a webpage in the browser.
5. In Wireshark, type `http` in the display filter field and click **Apply**.
6. After enough packets have been captured, select **Capture** → **Stop**.
7. Analyze the first HTTP GET request and its corresponding response message.

## OUTPUT

### (a) Source and Destination IP Address of the First GET Message

Select the first HTTP GET packet and expand the *Internet Protocol* section in the packet details pane. The IP address of the local machine can be confirmed using `hostname -I`.

**Source IP:** `192.168.1.17` **Destination IP:** `52.221.221.142`

### (b) Medium Format, Language, Encoding and Character Set Accepted by the Client

Expand the *Hypertext Transfer Protocol* section of the first GET packet and read the `Accept`, `Accept-Language`, `Accept-Encoding` and `Accept-Charset` fields.

- **Accept (medium format):** `text/html, application/xhtml+xml, application/xml; q = 0.9, */*; q= 0.8/r/n`
- **Accept-Language:** `en-US, en ; q=0.5/r/n`
- **Accept-Encoding:** `gzip, deflate/r/n`
- **Accept-Charset:** not present in this request

### (c) URL of the Website and User Agent

Read the `Host` and `User-Agent` fields in the headers of the GET request.

- **Host:** `nerverssl.com`
- **Full request URI:** `http://brightlushquietsunset.neverssl.com/`
- **User-Agent:** `Brave 1.47`

### (d) Source and Destination IP Address of the First Response Message

Locate the first HTTP response (`200 OK`) and expand the *Internet Protocol* section. The source and destination are reversed compared with the GET request.

**Source IP:** `52.221.221.142` **Destination IP:** `192.168.1.17`

### (e) Status Code of the First Response Message

Read the status code and reason phrase in the response line.

**Status Code:** `200` **Reason Phrase:** `OK`

### (f) Last Modified Time of the HTML File at the Server

Read the `Last-Modified` header field in the first response.

**Last-Modified:** `Mon, 17 Nov 2025 23:19:21 GMT`

### (g) Value of the Content-Length Field of the First Response

Read the `Content-Length` field in the response headers.

**Content-Length:** `2` bytes

### (h) Time Taken from GET Message to Response

Note the Time column value of the GET packet and of the corresponding response; the difference gives the response time. By default, the Time column shows seconds elapsed since the capture began.

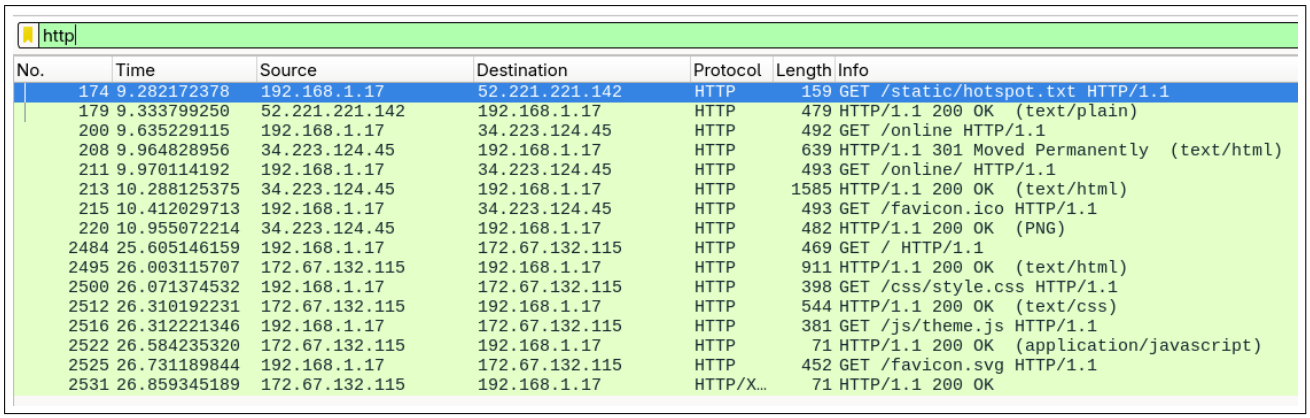
| GET (packet 174) | Response (packet 179) | Difference |
|---|---|---|
| 9.282172378 s | 9.333799250 s | 0.051626872 s |

The response was received about 0.0516 s (approximately 51.6 ms) after the GET message was sent.

### (i) HTTP Version Used by the Browser

Read the request line of the first GET packet.

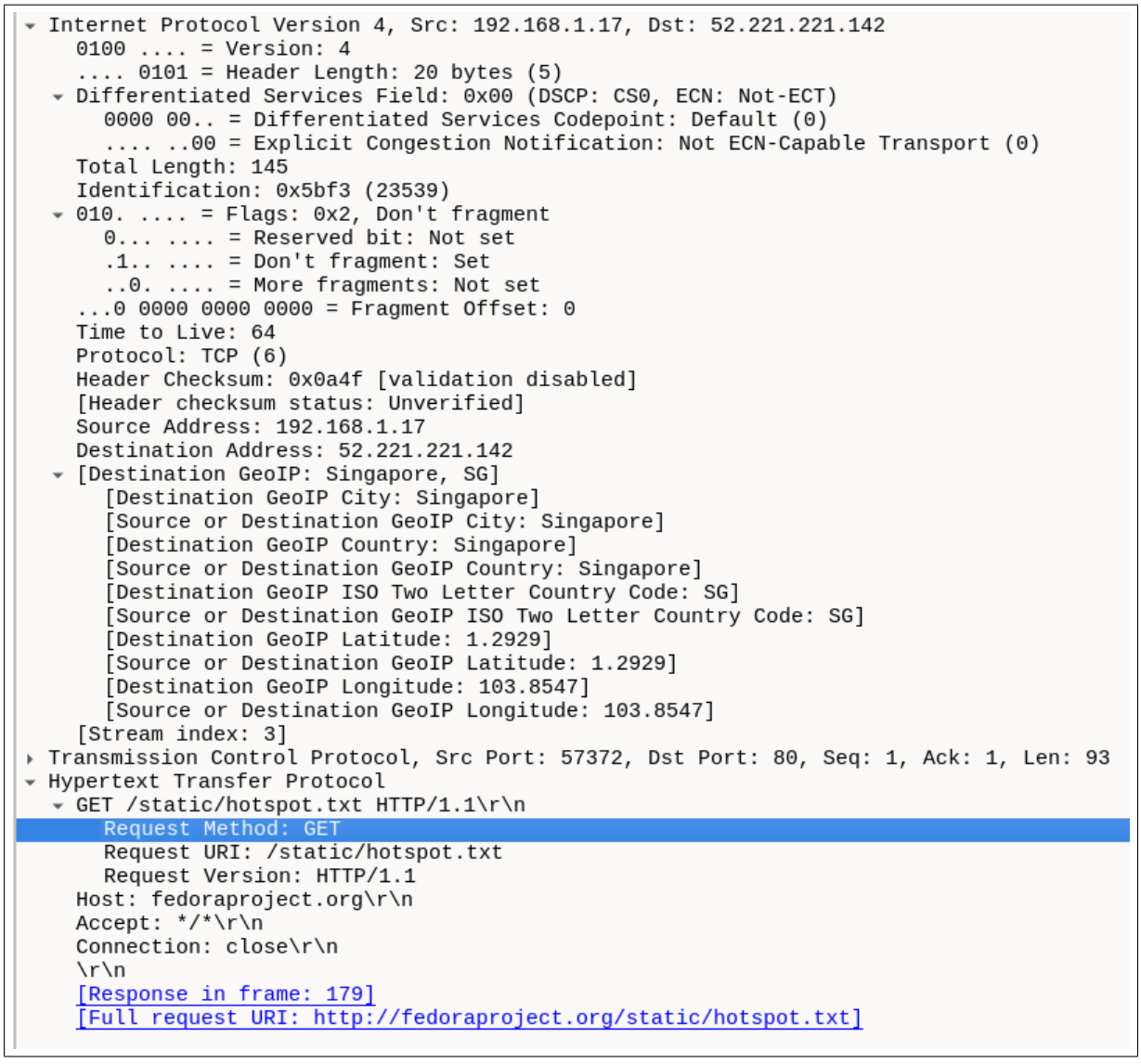
**HTTP Version:** `HTTP/1.1`



http

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 174 | 9.282172378 | 192.168.1.17 | 52.221.221.142 | HTTP | 159 | GET /static/hotspot.txt HTTP/1.1 |
| 179 | 9.333799250 | 52.221.221.142 | 192.168.1.17 | HTTP | 479 | HTTP/1.1 200 OK (text/plain) |
| 200 | 9.635229115 | 192.168.1.17 | 34.223.124.45 | HTTP | 492 | GET /online HTTP/1.1 |
| 208 | 9.964828956 | 34.223.124.45 | 192.168.1.17 | HTTP | 639 | HTTP/1.1 301 Moved Permanently (text/html) |
| 211 | 9.970114192 | 192.168.1.17 | 34.223.124.45 | HTTP | 493 | GET /online/ HTTP/1.1 |
| 213 | 10.288125375 | 34.223.124.45 | 192.168.1.17 | HTTP | 1585 | HTTP/1.1 200 OK (text/html) |
| 215 | 10.412029713 | 192.168.1.17 | 34.223.124.45 | HTTP | 493 | GET /favicon.ico HTTP/1.1 |
| 220 | 10.955072214 | 34.223.124.45 | 192.168.1.17 | HTTP | 482 | HTTP/1.1 200 OK (PNG) |
| 2484 | 25.605146159 | 192.168.1.17 | 172.67.132.115 | HTTP | 469 | GET / HTTP/1.1 |
| 2495 | 26.003115707 | 172.67.132.115 | 192.168.1.17 | HTTP | 911 | HTTP/1.1 200 OK (text/html) |
| 2500 | 26.071374532 | 192.168.1.17 | 172.67.132.115 | HTTP | 398 | GET /css/style.css HTTP/1.1 |
| 2512 | 26.310192231 | 172.67.132.115 | 192.168.1.17 | HTTP | 544 | HTTP/1.1 200 OK (text/css) |
| 2516 | 26.312221346 | 192.168.1.17 | 172.67.132.115 | HTTP | 381 | GET /js/theme.js HTTP/1.1 |
| 2522 | 26.584235320 | 172.67.132.115 | 192.168.1.17 | HTTP | 71 | HTTP/1.1 200 OK (application/javascript) |
| 2525 | 26.731189844 | 192.168.1.17 | 172.67.132.115 | HTTP | 452 | GET /favicon.svg HTTP/1.1 |
| 2531 | 26.859345189 | 172.67.132.115 | 192.168.1.17 | HTTP/X… | 71 | HTTP/1.1 200 OK |

Figure 16: Wireshark capture with `http` display filter applied



```
▾ Internet Protocol Version 4, Src: 192.168.1.17, Dst: 52.221.221.142
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▾ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      0000 00.. = Differentiated Services Codepoint: Default (0)
      .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 145
    Identification: 0x5bf3 (23539)
  ▾ 010. .... = Flags: 0x2, Don't fragment
      0... .... = Reserved bit: Not set
      .1.. .... = Don't fragment: Set
      ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x0a4f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.1.17
    Destination Address: 52.221.221.142
  ▾ [Destination GeoIP: Singapore, SG]
      [Destination GeoIP City: Singapore]
      [Source or Destination GeoIP City: Singapore]
      [Destination GeoIP Country: Singapore]
      [Source or Destination GeoIP Country: Singapore]
      [Destination GeoIP ISO Two Letter Country Code: SG]
      [Source or Destination GeoIP ISO Two Letter Country Code: SG]
      [Destination GeoIP Latitude: 1.2929]
      [Source or Destination GeoIP Latitude: 1.2929]
      [Destination GeoIP Longitude: 103.8547]
      [Source or Destination GeoIP Longitude: 103.8547]
    [Stream index: 3]
▸ Transmission Control Protocol, Src Port: 57372, Dst Port: 80, Seq: 1, Ack: 1, Len: 93
▾ Hypertext Transfer Protocol
  ▾ GET /static/hotspot.txt HTTP/1.1\r\n
      Request Method: GET
      Request URI: /static/hotspot.txt
      Request Version: HTTP/1.1
    Host: fedoraproject.org\r\n
    Accept: */*\r\n
    Connection: close\r\n
    \r\n
    [Response in frame: 179]
    [Full request URI: http://fedoraproject.org/static/hotspot.txt]
```

Figure 17: Details of the first HTTP GET request (packet 174)

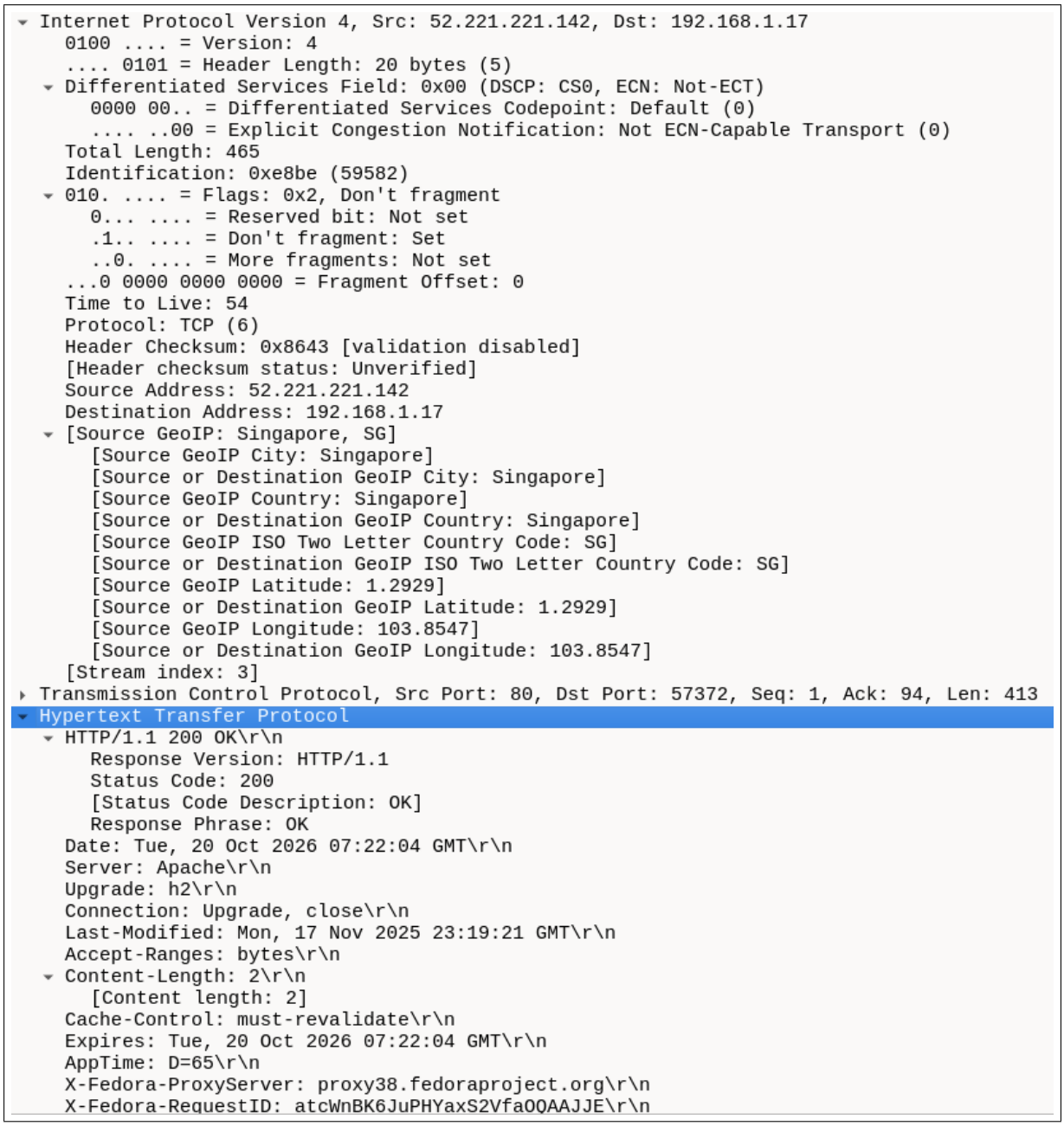



Figure 18: Details of the first HTTP response (packet 179)

## RESULT

HTTP packets were successfully captured using Wireshark. The first GET request and its response were analyzed to identify the source and destination IP addresses, request headers, response status code (200 OK), Last-Modified time, Content-Length, response time (about 0.0516 s) and the HTTP version (HTTP/1.1).

# Experiment No. 3

# CAPTURING AND ANALYZING SMTP PACKETS USING WIRESHARK

---

## AIM

Compose an e-mail and address it to yourself, but do not send it yet. Open the Wireshark and start capturing. Go to your e-mail user agent and send the e-mail. In the Wireshark window, type `smtp` in the filter field and click Apply. Stop capturing and save the captured file. Using the captured information, answer the following:

(a) All SMTP packets have the same two IP addresses. Which one is the IP address of your computer? Which host does the other IP address represent?

(b) All SMTP packets have the same two port numbers. Which one is the port number of the SMTP client process? In which range is the client port number?

(c) What is the port number of the SMTP server process?

(d) Examine the SMTP commands or SMTP response codes in each SMTP packet and write down their meanings.

(e) There is an IMF packet that is encapsulated inside an SMTP packet. What is the content of this packet?

## PROCEDURE

1. Open Thunderbird, compose an e-mail addressed to yourself, but do not send it yet.
2. Open Wireshark, select the active network interface and click **Start Capturing Packets**.
3. Return to Thunderbird and send the e-mail.
4. In Wireshark, type `smtp` in the display filter field and click **Apply**.
5. Select **Capture** → **Stop** and save the captured file.
6. Analyze the captured SMTP packets.

## OUTPUT

### (a) IP Addresses

All SMTP packets carry the same two IP addresses: one belongs to the local computer (SMTP client) and the other to the mail server.

**IP of my computer:** `192.168.1.17` **IP of the other host:** `192.179.20.109`

### (b) Client Port Number

The client port is assigned dynamically by the operating system from the ephemeral (dynamic) port range, which is used for temporary connections initiated by client applications.

**SMTP client port: 33856**

### (c) SMTP Server Port Number

**SMTP server port: 587**

### (d) SMTP Commands and Response Codes

| Command / Code | Meaning |
|---|---|
| `EHLO` | Extended HELO command used by the client to introduce itself and to ask for the server's capabilities. |
| `MAIL FROM` | Specifies the sender's e-mail address. |
| `220` | Service ready; the server is ready to accept the connection. |
| `250` | Requested action completed ("at your service"); the server's reply to `EHLO`, listing its capabilities. |
| `502` | Unrecognized command; the server did not understand what it received. |
| `530` | Authentication required. |

Table 1: SMTP commands and response codes observed

### (e) Content of the IMF Packet

The Internet Message Format (IMF) packet is encapsulated inside the SMTP `DATA` exchange and carries the message headers and body. In this capture the IMF packet cannot

be found, because the connection to the Gmail server is encrypted. The message content (From, To, Subject and body) is therefore not visible in Wireshark.

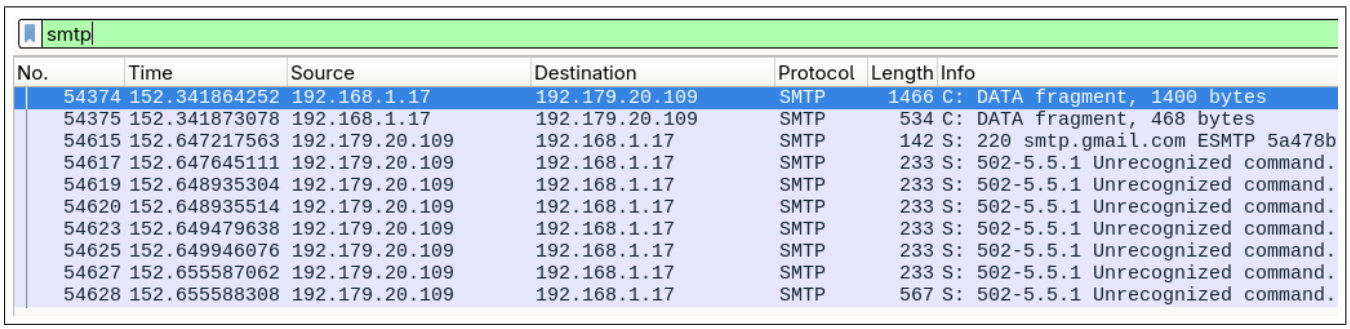



Figure 19: Wireshark capture with `smtp` display filter applied

## RESULT

SMTP packets were successfully captured and analyzed using Wireshark. The IP addresses, client and server port numbers, SMTP commands and response codes were identified and studied.

# Experiment No. 4

# CAPTURING AND ANALYZING DNS PACKETS USING WIRESHARK

---

**AIM**

First, clear the DNS record from the cache memory of your computer. For this, use `ipconfig/flushdns` on Windows or `systemd-resolve --flush-caches` on Linux. Next, clear your browser's cache memory. Open the Wireshark and start capturing. In your browser visit your college website. Wireshark starts to capture packets. Type `dns` in the filter field and press Apply so that only DNS messages are displayed. Stop capturing and save the captured file. Using the captured information, answer the following questions:

(a) Locate the first DNS query message resolving your college website. What is the packet number (this "packet number" is assigned by Wireshark for listing purposes only; it is NOT a packet number contained in any real packet header) in the trace for the DNS query message?

(b) Is this query message sent over UDP or TCP?

(c) Now locate the corresponding DNS response to the initial DNS query. What is the packet number in the trace for the DNS response message? Is this response message received via UDP or TCP?

(d) What are the source and destination port numbers for the DNS query message?

(e) What are the source and destination port numbers for the DNS response message?

(f) To what IP address is the DNS query message sent?

(g) What is the query message ID number? What is the response message ID number? What is the purpose of this field?

(h) What is the length of the flag field in a DNS message?

(i) Which bit in the flag field determines whether the message is a query or a response?

(j) Which bits are used only in the response message? What is the function of these bits in the response message?

(k) How many question records, answer records, authority records, and additional records are present in the query message?

(l) How many question records, answer records, authority records, and additional records are present in the response message?

## PROCEDURE

1. Open Wireshark and select the active network interface (Wi-Fi or Ethernet).
2. Clear the DNS cache of the system so that a fresh lookup is generated:

```
sudo resolvectl flush-caches
```

3. Clear the browser cache.
4. Click **Start Capturing Packets**.
5. Visit the college website (`cet.ac.in`) in the browser.
6. Type `dns` in the display filter field and click **Apply**.
7. Select **Capture** → **Stop**.
8. Save the capture file using **File** → **Save As** with a `.pcapng` extension.
9. Analyze the DNS query and response packets.

## OUTPUT

### (a) Packet Number of the First DNS Query

**Packet No.:** `47`

### (b) Transport Protocol of the Query (UDP or TCP)

**Protocol:** `UDP (User Datagram Protocol)`

### (c) Packet Number and Protocol of the DNS Response

**Packet No.:** `61` (for type `A` query `0xd1ef`) / `63` (for `0x7084`) **Protocol:** `UDP`

### (d) Source and Destination Ports of the DNS Query

**Source Port:** `35011` (Packet 47) / `42308` (Packet 49) **Destination Port:** `53`

### (e) Source and Destination Ports of the DNS Response

**Source Port:** `53` **Destination Port:** `42308` (Packet 61) / `35011` (Packet 63)

### (f) Destination IP Address of the DNS Query

**Destination IP:** `10.219.57.53` (IPv4 DNS Server)

### (g) Query and Response Message ID (Transaction ID)

**Query ID:** `0x7084` (Packet 47) / `0xd1ef` (Packet 49) **Response ID:** `0xd1ef` (Packet 61) / `0x7084` (Packet 63)

The transaction ID is used to match a response with the query that produced it. The server copies the ID of the query into its response.

### (h) Length of the Flags Field

The Flags field in a DNS message is **16 bits (2 bytes)** long (`0x0100` in the query and `0x8180` in the response).

### (i) Bit that Distinguishes a Query from a Response

The **QR** (1st / most significant) bit in the Flags field is 0 for a query and 1 for a response.

**QR value in the query:** `0` **QR value in the response:** `1`

### (j) Bits Used Only in the Response Message

- **AA (Authoritative Answer):** set when the responding server is authoritative for the queried domain.
- **RA (Recursion Available):** set when the server supports recursive queries.
- **RCODE (Response Code):** indicates the status of the response (e.g. 0 = no error, 3 = name does not exist).

**AA:** `0` (Not authoritative) **RA:** `1` (Recursion available) **RCODE:** `0` (No error)

### (k) and (l) Resource Record Counts

| Message | Questions | Answer RRs | Authority RRs | Additional RRs |
|---|---|---|---|---|
| Query (Packet 47 / 49) | 1 | 0 | 0 | 1 |
| Response (Packet 61) | 1 | 1 | 0 | 1 |

Table 2: Number of records in the DNS query and response for `cet.ac.in`

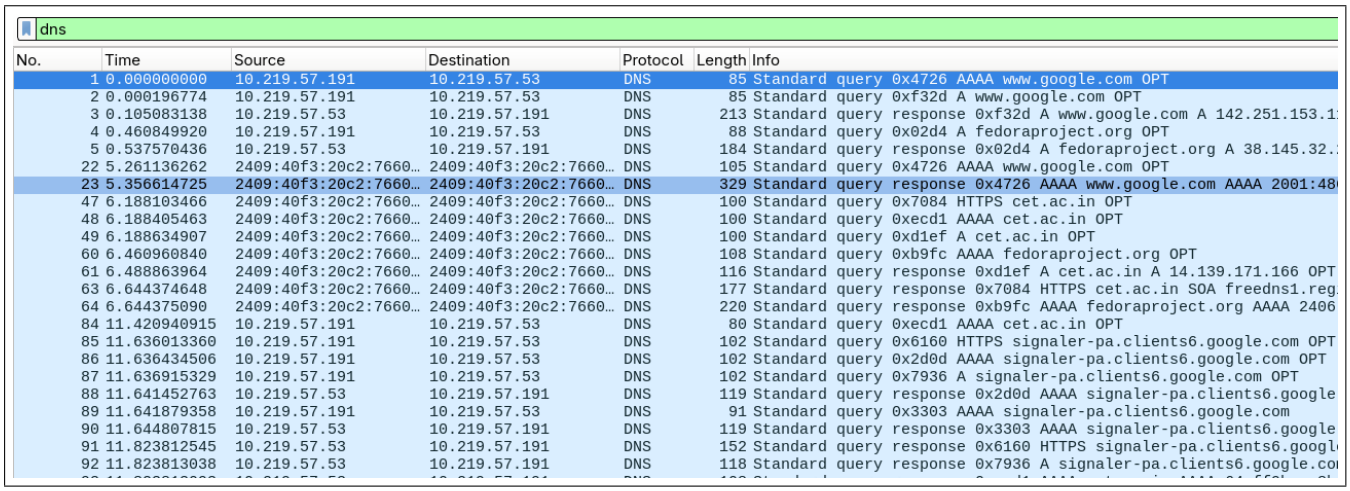



Figure 20: Wireshark capture with `dns` display filter applied

```
▾ User Datagram Protocol, Src Port: 35011, Dst Port: 53
    Source Port: 35011
    Destination Port: 53
    Length: 46
    Checksum: 0x496d [unverified]
    [Checksum Status: Unverified]
    [Stream index: 5]
    [Stream Packet Number: 1]
  ▾ [Timestamps]
      [Time since first frame: 0.000000000 seconds]
      [Time since previous frame: 0.000000000 seconds]
    UDP payload (38 bytes)
▾ Domain Name System (query)
    Transaction ID: 0x7084
  ▾ Flags: 0x0100 Standard query
      0... .... .... .... = Response: Message is a query
      .000 0... .... .... = Opcode: Standard query (0)
      .... ..0. .... .... = Truncated: Message is not truncated
      .... ...1 .... .... = Recursion desired: Do query recursively
      .... .... .0.. .... = Z: reserved (0)
      .... .... ...0 .... = Non-authenticated data: Unacceptable
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 1
  ▾ Queries
    ▾ cet.ac.in: type HTTPS, class IN
        Name: cet.ac.in
        [Name Length: 9]
        [Label Count: 3]
        Type: HTTPS (65) (HTTPS Specific Service Endpoints)
        Class: IN (0x0001)
  ▾ Additional records
    ▾ <Root>: type OPT
        Name: <Root>
        Type: OPT (41)
        UDP payload size: 1452
        Higher bits in extended RCODE: 0x00
        EDNS0 version: 0
      ▾ Z: 0x0000
          0... .... .... .... = DO bit: Cannot handle DNSSEC security RRs
          .000 0000 0000 0000 = Reserved: 0x0000
        Data length: 0
    [Response In: 63]
```

Figure 21: Details of the DNS query packet for `cet.ac.in`




Figure 22: Details of the DNS response packet for `cet.ac.in`

## RESULT

DNS packets for `cet.ac.in` were successfully captured and analyzed using Wireshark. The DNS query and response packets, source and destination IP addresses, port numbers, transaction IDs, flags and resource records were identified and studied.

# Experiment No. 5

# TCP MATRIX TYPE IDENTIFICATION

---

## AIM

Client-Server communication using TCP: The client inputs an integer N and creates a square matrix of order N by populating the matrix with random numbers in the range [1,50]. It then sends the matrix to the server which identifies the matrix type (upper triangular, lower triangular, diagonal). The server then informs the type (as a string) to the client which it prints.

## ALGORITHM

### Server Algorithm

1. Start.
2. Initialize a TCP socket and bind it to a specific IP address and port (bind()).
3. Listen for incoming client connection requests (listen()).
4. Accept a connection request from a client, establishing a dedicated communication socket (accept()).
5. Receive the order $N$ and the matrix elements sent by the client (recv() / read()).
6. Process the Matrix to determine its type using the following checks:
   - Diagonal Matrix: Check if all elements where row index $i \neq$ column index $j$ are equal to zero.
   - Upper Triangular Matrix: Check if all elements below the main diagonal ($i > j$) are equal to zero.
   - Lower Triangular Matrix: Check if all elements above the main diagonal ($i < j$) are equal to zero.
7. Format the result as a string (e.g., "Upper Triangular Matrix").
8. Send the result string back to the client via the socket (send() / write()).
9. Close the client connection socket.
10. Stop.

**Client Algorithm**

1. Start.
2. Initialize a TCP socket using socket programming functions (socket()).
3. Connect to the server using the server's IP address and port number (connect()).
4. Input the integer $N$ representing the order of the square matrix from the user.
5. Generate an $N \times N$ matrix and populate it with random numbers in the range $[1, 50]$.
6. Display the generated matrix on the client console.
7. Send the order $N$ followed by the matrix elements to the server via the TCP connection (send() / write()).
8. Receive the matrix type string sent back by the server (recv() / read()).
9. Print the received matrix type on the client console.
10. Close the socket connection (close()).
11. Stop.

**PROGRAM**

**Server**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int port;
int checkMatrix(int n, int matrix[][n]){
        int isUpper = 1, isLower = 1, isDiagonal = 1;
        for(int i = 0; i < n; i++){
                for(int j = 0; j < n;j++){
                        if(i < j && matrix[i][j] != 0) isLower = 0;
                        if(i > j && matrix[i][j] != 0) isUpper = 0;
                        if(i != j && matrix[i][j] != 0) isDiagonal = 0;
                }
```

```
        }
        if(isDiagonal) return 2;
        else if(isUpper) return 0;
        else if(isLower) return 1;
}
void display(int n, int matrix[][n]) {
        for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                        printf("%d ", matrix[i][j]);
                } printf("\n");
        }
}
int main() {
    int sockFD = socket(AF_INET, SOCK_STREAM, 0);
    if (sockFD < 0) {
        perror("Socket creation failed");
        return 1;
    }

    struct sockaddr_in address, client;
    socklen_t len = sizeof(client);
    printf("Enter Port: "); scanf("%d", &port);
    char buffer[1024];
    address.sin_family = AF_INET;
    address.sin_port = htons(port);
    address.sin_addr.s_addr = INADDR_ANY;

    if (bind(sockFD, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        close(sockFD);
        return 1;
    }
    listen(sockFD, 5);
    printf("Waiting for client...\n");
    int newsock = accept(sockFD, (struct sockaddr *)&client, &len);
    if (newsock < 0) {
        perror("Accept failed");
        close(sockFD);
        return 1;
    }
    memset(buffer, 0, sizeof(buffer));
```

```
    read(newsock, buffer, sizeof(buffer));
    printf("Message: %s\n", buffer);
    send(newsock, "Hello from server", 17, 0);
    int n;
    read(newsock, &n, sizeof(n));
    int matrix[n][n];
    read(newsock, &matrix, sizeof(matrix));

    printf("\nMatrix recieved from Client:\n");
    display(n, matrix);
    int result = checkMatrix(n, matrix);
    char output[1024];
    if(result == 0) strcpy(output, "Upper Triangle Matrix");
    else if(result == 1) strcpy(output, "Lower Triangle Matrix");
    else strcpy(output,"Diagonal Matrix");
    printf("\nSending output: \"%s\" to Client\n", output);

    send(newsock, output, strlen(output), 0);
    close(newsock);
    close(sockFD);
    printf("\nClient disconnected...\n");
    return 0;
}
```

**Client**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>

int port;
void fillLower(int n, int matrix[][n]);
void fillDiagonal(int n, int matrix[][n]);
void fillUpper(int n, int matrix[][n]);
void fillRandom(int n, int matrix[][n]);
```

```
void display(int n, int matrix[][n]);

void fillUpper(int n, int matrix[][n]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (j >= i)
                matrix[i][j] = rand() % 50 + 1;
            else
                matrix[i][j] = 0;
        }
    }
}
void fillLower(int n, int matrix[][n]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (j <= i)
                matrix[i][j] = rand() % 50 + 1;
            else
                matrix[i][j] = 0;
        }
    }
}
void fillDiagonal(int n, int matrix[][n]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                matrix[i][j] = rand() % 50 + 1;
            else
                matrix[i][j] = 0;
        }
    }
}
void fillRandom(int n, int matrix[][n]) {
    int choose = rand() % 3;
    if (choose == 0)
        fillUpper(n, matrix);
    else if (choose == 1)
        fillLower(n, matrix);
    else
        fillDiagonal(n, matrix);
}
```

```
void display(int n, int matrix[][n]) {
    printf("\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%3d ", matrix[i][j]);
        }
        printf("\n");
    }
}
int main() {
    int sockFD = socket(AF_INET, SOCK_STREAM, 0);
    if (sockFD < 0) {
        perror("Socket creation failed");
        return 1;
    }

    struct sockaddr_in address;
    printf("Enter Port: "); scanf("%d", &port);
    address.sin_family = AF_INET;
    address.sin_port = htons(port);
    address.sin_addr.s_addr = inet_addr("127.0.0.1");

    int status = connect(sockFD, (struct sockaddr *)&address,
    sizeof(address));
    if (status < 0) {
        perror("Connection failed");
        close(sockFD);
        return 1;
    }

    printf("Connection was successful...\n");
    char buffer[1024] = "Hello from client";
    send(sockFD, buffer, strlen(buffer), 0);
    memset(buffer, 0, sizeof(buffer));
    read(sockFD, buffer, sizeof(buffer));
    printf("Message from Server: %s\n", buffer);
    memset(buffer, 0, sizeof(buffer));

    srand(time(NULL));
    printf("Enter N = "); int n;
    scanf("%d", &n);
```

```
    int matrix[n][n];
    fillRandom(n, matrix);

    printf("\nMatrix Sent from Client: \n");
    display(n, matrix);
    send(sockFD, &n, sizeof(n), 0);
    send(sockFD, matrix, sizeof(matrix), 0);
    char output[1024];
    read(sockFD, output, sizeof(output));
    printf("\nMessage from Server: %s\n", output);

    close(sockFD);
    return 0;
}
```

## OUTPUT

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/5_Matrix_Type_Identification$ ./server
Enter Port: 8088
Waiting for client...
Message: Hello from client

Matrix recieved from Client:
27 0 0 0
0 45 0 0
0 0 10 0
0 0 0 40

Sending output: "Diagonal Matrix" to Client

Client disconnected...
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/5_Matrix_Type_Identification$
```

Figure 23: Terminal Output (Server)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/5_Matrix_Type_Identification$ ./client
Enter Port: 8088
Connection was successful...
Message from Server: Hello from server
Enter N = 4

Matrix Sent from Client:

 27   0   0   0
  0  45   0   0
  0   0  10   0
  0   0   0  40

Message from Server: Diagonal Matrix
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/5_Matrix_Type_Identification$
```

Figure 24: Terminal Output (Client)

## RESULT

The program was executed and verified successfully.

# Experiment No. 6

# UDP NEW-GENERATION ENGLISH TRANSLATOR

---

## AIM

Client-Server communication using UDP: You are very good at communicating in the "new generation" English language with all sorts of abbreviations like tbh, ig, etc. Now design a client-server application as follows: The client inputs a new-generation English sentence from the user and sends it to the server. The server then translates the received sentence to formal English and sends the translated sentence back to the client which it prints.

*Sample string sent to the server*

```
Really idc about this stupid server as it is of no use irl but atm,
I will design one, tbf to the professor.
```

*Translated string sent back to the client*

```
Really I don't care about this stupid server as it is of no use in
real life but at the moment, I will design one, to be fair to the
professor.
```

Only the following abbreviations are considered: `tbh, ig, tbf, atm, irl, lol, asap, omg, ttyl, idk, nvm`

## ALGORITHM

### Server Algorithm

1. Start.
2. Initialize a UDP socket using socket().
3. Bind the socket to a designated IP address and port number using the bind() function.

4. Display a status message indicating that the server is ready and listening for incoming UDP packets.
5. Receive the incoming message packet and the client's address tuple (IP, port) using recvfrom().
6. Extract and decode the byte data into a string.
7. Process the Text (Translation Logic):
    - Split the incoming sentence into individual words.
    - Maintain a predefined dictionary mapping abbreviations to their formal equivalents (e.g., "irl": "in real life", "atm": "at the moment").
    - Iterate through each word, strip trailing punctuation, and check if it matches any key in the abbreviation dictionary (case-insensitive).
    - If a match is found, replace the abbreviation with its formal expansion while preserving punctuation.
    - Recombine the words into a single translated formal sentence.
8. Send the translated formal string back to the client's address using the sendto() function.
9. Stop.

**Client Algorithm**

1. Start.
2. Initialize a UDP socket using socket().
3. Prompt the user to input a sentence containing modern abbreviations (e.g., *idc*, *irl*, *atm*, *tbf*).
4. Send the input sentence as a byte stream to the server's IP address and port using the sendto() function.
5. Wait for the response packet from the server using the recvfrom() function.
6. Receive the translated formal English string from the server.
7. Print the translated string on the client console.
8. Close the socket connection (close()).
9. Stop.

## PROGRAM

### Server

```
#include <arpa/inet.h>
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>
#include <unistd.h>
#include <string.h>

#define BUFFER_SIZE 1024
int port;
void translate(char* buffer){
        char result[1024] = {0}, word[100];
        int i = 0, j = 0;
        while(buffer[i] != '\0'){
                if(isalpha(buffer[i])){
                        j = 0;
                        while(isalpha(buffer[i])){
                                word[j++] = buffer[i++];
                        }
                        word[j] = '\0'; i--;
                        if(strcmp(word, "tbh") == 0){
                                strcpy(word, "to be honest");
                        } else if(strcmp(word, "ig") == 0){
                                strcpy(word, "I guess");
                        } else if(strcmp(word, "tbf") == 0){
                                strcpy(word, "to be fair");
                        } else if(strcmp(word, "atm") == 0){
                                strcpy(word, "at the moment");
                        } else if(strcmp(word, "irl") == 0){
                                strcpy(word, "in real life");
                        } else if(strcmp(word, "lol") == 0){
                                strcpy(word, "laugh out loud");
                        } else if(strcmp(word, "asap") == 0){
                                strcpy(word, "as soon as possible");
                        } else if(strcmp(word, "omg") == 0){
                                strcpy(word, "oh my God");
                        } else if(strcmp(word, "ttyl") == 0){
                                strcpy(word, "talk to you later");
```

```
                        } else if(strcmp(word, "idk") == 0){
                                strcpy(word, "I don't know");
                        } else if(strcmp(word, "nvm") == 0){
                                strcpy(word, "nevermind");
                        }
                        strcat(result, word);
                } else {
                        result[strlen(result)] = buffer[i];
                        result[strlen(result)+1] = '\0';
                }
                i++;
        }
        result[strlen(result)] = '\0';
        strcpy(buffer, result);
}
int main(){
        int sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
        if(sock_fd < 0){
                printf("Socket Creation Failed !\n");
                return 1;
        }
        struct sockaddr_in address, client_addr;
        socklen_t client_len = sizeof(client_addr);

        printf("Enter port: "); scanf("%d", &port);
        char buffer[1024];
        address.sin_family = AF_INET;
        address.sin_port = htons(port);
        address.sin_addr.s_addr = INADDR_ANY;

        if(bind(sock_fd, (struct sockaddr *)&address, sizeof(address)) < 0){
                printf("Bind Failed !\n");
                close(sock_fd);
                return 1;
        }
        printf("\nWaiting for Client....\n");
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sock_fd, buffer, BUFFER_SIZE, 0,
                 (struct sockaddr *)&client_addr, &client_len);
        printf("\nMessage from client: %s\n", buffer);
        sendto(sock_fd, "Hello from server", 17, 0,
```

```
               (struct sockaddr *)&client_addr, client_len);
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sock_fd, buffer, BUFFER_SIZE, 0,
                 (struct sockaddr *)&client_addr, &client_len);
        printf("\nMessage from client: \"%s\"\n", buffer);

        translate(buffer);
        buffer[strlen(buffer)] = '\0';
        printf("\nTranslated message sent to client: \"%s\"\n", buffer);
        sendto(sock_fd, buffer, strlen(buffer) + 1, 0,
               (struct sockaddr *)&client_addr, client_len);
        close(sock_fd);
        printf("\nClient disconnected...\n");
}
```

### Client

```
#include <stdio.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>
#include <sys/socket.h>

#define BUFFER_SIZE 1024
int port;
int main(){
        char buffer[1024];
        struct sockaddr_in server_addr;
        socklen_t server_len = sizeof(server_addr);
        int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
        printf("Enter port: "); scanf("%d", &port); getchar();

        server_addr.sin_family = AF_INET;
        server_addr.sin_port = htons(port);
        server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
        sendto(sockfd, "Hello from Client...", 20, 0,
           (struct sockaddr *)&server_addr, server_len);
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
             (struct sockaddr *)&server_addr, &server_len);
        printf("Message from server: %s\n", buffer);
```

```
        memset(buffer, 0, sizeof(buffer));
        printf("Enter new-gen sentence: ");
        scanf("%[^\n]s", buffer);

        printf("\nInputted sentence: %s\n", buffer);
        sendto(sockfd, buffer, strlen(buffer) + 1, 0,
           (struct sockaddr *)&server_addr, server_len);
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
             (struct sockaddr *)&server_addr, &server_len);
        printf("Translated sentence from server: %s\n", buffer);
        close(sockfd);
        return 0;
}
```

## OUTPUT

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/6_String_Identification$ ./server
Enter port: 5055

Waiting for Client....

Message from client: Hello from Client...

Message from client: "tbh idk what does this mean, nvm."

Translated message sent to client: "to be honest I don't know what does this mean, nevermind."

Client disconnected...
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/6_String_Identification$
```

Figure 25: Terminal Output (Server)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/6_String_Identification$ ./client
Enter port: 5055
Message from server: Hello from server
Enter new-gen sentence: tbh idk what does this mean, nvm.

Inputted sentence: tbh idk what does this mean, nvm.
Translated sentence from server: to be honest I don't know what does this mean, nevermind.
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/6_String_Identification$
```

Figure 26: Terminal Output (Client)

## RESULT

The program was executed and verified successfully.

# Experiment No. 7

# TCP MULTI-USER CHAT SERVER

---

### AIM

Implement a multi-user chat server using TCP as the transport layer protocol.

### ALGORITHM

#### Server Algorithm

1. Start.
2. Create Server Socket using socket().
3. Bind and associate the server socket with an IP address and a predefined port number using bind().
4. Put the server socket into listening mode using listen().
5. Initialize client list by creating a data structure (e.g., a list or set) to keep track of all active client sockets and their threads.
6. Accept Connections (Main Loop):
    - Continuously wait for incoming client connection requests using accept().
    - Upon a successful connection, accept the client socket and create a new dedicated Client Handler Thread for that client.
    - Add the client socket to the active client list.
7. Handle Client Communication (Inside Client Thread):
    - Receive: Continuously read incoming message bytes from the client socket.
    - Broadcast: When a message is received, iterate through the active client list and send the message to all other connected client sockets (excluding the sender).
    - Disconnection: If the client disconnects, close the client socket, remove it from the active client list, terminate the thread, and broadcast a departure notification.

8. Stop.

### Client Algorithm

1. Start.
2. Create client socket using socket().
3. Establish a connection to the server's IP address and port using connect().
4. Spawn dual channels (Concurrency), by spliting execution into two concurrent tasks:
   - Receiver Thread:
     - Continuously listen for incoming data from the client_socket.
     - Print received messages onto the user's console.
   - Sender Loop (Main Thread):
     - Read chat input from the user's console (stdin).
     - Send the typed message across the client_socket to the server.
5. Disconnect: If the user types an exit command, close the client_socket and terminate the program.
6. Stop.

## PROGRAM

### Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <arpa/inet.h>

#define MAX_CLIENTS 10
#define BUFFER_SIZE 1024
typedef struct {
    int client;
    char* client_name;
```

```
} clients;

clients client_sockets[MAX_CLIENTS];
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void broadcast_message(char *msg, int sender_fd) {
    pthread_mutex_lock(&lock);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (client_sockets[i].client != 0 && client_sockets[i].client !=
   sender_fd) {
            send(client_sockets[i].client, msg, strlen(msg), 0);
        }
    }
    pthread_mutex_unlock(&lock);
}
void* handle_client(void* arg) {
    int client_fd = *(int*)arg;
    free(arg);
    char buffer[BUFFER_SIZE];
    char name_buffer[64];
    memset(name_buffer, 0, sizeof(name_buffer));
    int name_bytes = recv(client_fd, name_buffer, sizeof(name_buffer) - 1, 0);
    if (name_bytes <= 0) {
        close(client_fd);
        return NULL;
    }
    pthread_mutex_lock(&lock);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (client_sockets[i].client == client_fd) {
            client_sockets[i].client_name = strdup(name_buffer);
            break;
        }
    }
    pthread_mutex_unlock(&lock);
    char join_msg[BUFFER_SIZE];
    snprintf(join_msg, sizeof(join_msg), "[Server] %s has joined the chat.",
   name_buffer);
    printf("%s\n", join_msg);
    broadcast_message(join_msg, client_fd);
    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
```

```
        if (bytes_received <= 0) {
            char leave_msg[BUFFER_SIZE];
            snprintf(leave_msg, sizeof(leave_msg), "[Server] %s has left the
    chat.", name_buffer);
            printf("%s\n", leave_msg);

            pthread_mutex_lock(&lock);
            for (int i = 0; i < MAX_CLIENTS; i++) {
                if (client_sockets[i].client == client_fd) {
                    client_sockets[i].client = 0;
                    free(client_sockets[i].client_name);
                    client_sockets[i].client_name = NULL;
                    break;
                }
            }
            pthread_mutex_unlock(&lock);
            broadcast_message(leave_msg, client_fd);
            close(client_fd);
            break;
        }
        char formatted_msg[BUFFER_SIZE + 80];
        snprintf(formatted_msg, sizeof(formatted_msg), "%s: %s", name_buffer,
    buffer);
        printf("%s\n", formatted_msg);
        broadcast_message(formatted_msg, client_fd);
    }
    return NULL;
}
int main() {
    int server_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("Socket creation failed");
        exit(1);
    }
    int PORT;
    printf("Enter port: ");
    scanf(" %d", &PORT);
```

```
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);
if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr))
< 0) {
    perror("Bind failed");
    exit(1);
}
if (listen(server_fd, MAX_CLIENTS) < 0) {
    perror("Listen failed");
    exit(1);
}

printf("Server is listening...\n");
while (1) {
    int client_fd = accept(server_fd, (struct sockaddr*)&client_addr,
&addr_len);
    if (client_fd < 0) {
        perror("Accept failed");
        continue;
    }
    pthread_mutex_lock(&lock);
    int i;
    for (i = 0; i < MAX_CLIENTS; i++) {
        if (client_sockets[i].client == 0) {
            client_sockets[i].client = client_fd;
            client_sockets[i].client_name = NULL;
            break;
        }
    }
    pthread_mutex_unlock(&lock);
    if (i == MAX_CLIENTS) {
        printf("Max clients reached! Connection rejected.\n");
        close(client_fd);
    } else {
        int *new_sock = malloc(sizeof(int));
        *new_sock = client_fd;
        pthread_t tid;
        pthread_create(&tid, NULL, handle_client, new_sock);
        pthread_detach(tid);
    }
```

```
    }
    close(server_fd);
    return 0;
}
```

### Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>

#define BUFFER_SIZE 1024
int sock;
struct sockaddr_in addr;
void* handle_chat(void *varg) {
    char buff[BUFFER_SIZE];
    while (1) {
        memset(buff, 0, sizeof(buff));
        int valread = read(sock, buff, BUFFER_SIZE - 1);
        if (valread <= 0) {
            printf("\nDisconnected from server.\n");
            exit(0);
        }
        printf("%s\n", buff);
        printf("> ");
        fflush(stdout);
    }
    return NULL;
}
int main() {
    int PORT;
    printf("Enter PORT: ");
    scanf(" %d", &PORT);
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        perror("Socket failed to open");
        exit(1);
    }
```

```
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) {
        perror("Invalid address");
        exit(1);
    }
    if (connect(sock, (struct sockaddr*) &addr, sizeof(addr)) < 0) {
        perror("Connect failed");
        exit(1);
    }

    char username[64];
    printf("Enter your username: ");
    scanf(" %63s", username);
    send(sock, username, strlen(username), 0);
    pthread_t th_id;
    pthread_create(&th_id, NULL, handle_chat, NULL);
    while (1) {
        char msg[BUFFER_SIZE];
        printf("> ");
        scanf(" %[^\n]", msg);
        send(sock, msg, strlen(msg), 0);
    }
    pthread_join(th_id, NULL);
    close(sock);
    return 0;
}
```

## OUTPUT

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$ ./server
Enter port: 5173
Server is listening...
[Server] 01_client has joined the chat.
[Server] 02_client has joined the chat.
01_client: Hello !
[Server] 03_client has joined the chat.
03_client: Hey chat, do we have CN today ?
02_client: yeah 5th hour
01_client: actually we've viva for cn
03_client: Ooh, all the best then
02_client: thanks..
01_client: thanks..bye
[Server] 01_client has left the chat.
[Server] 02_client has left the chat.
[Server] 03_client has left the chat.
^C
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$
```

Figure 27: Terminal Output (Server)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$ ./client
Enter PORT: 5173
Enter your username: 01_client
> [Server] 02_client has joined the chat.
> Hello !
> [Server] 03_client has joined the chat.
> 03_client: Hey chat, do we have CN today ?
> 02_client: yeah 5th hour
> actually we've viva for cn
> 03_client: Ooh, all the best then
> 02_client: thanks..
> thanks..bye
> ^C
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$
```

Figure 28: Terminal Output (Client 1)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$ ./client
Enter PORT: 5173
Enter your username: 02_client
> 01_client: Hello !
> [Server] 03_client has joined the chat.
> 03_client: Hey chat, do we have CN today ?
> yeah 5th hour
> 01_client: actually we've viva for cn
> 03_client: Ooh, all the best then
> thanks..
> 01_client: thanks..bye
> [Server] 01_client has left the chat.
> ^C
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$
```

Figure 29: Terminal Output (Client 2)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$ ./client
Enter PORT: 5173
Enter your username: 03_client
> Hey chat, do we have CN today ?
> 02_client: yeah 5th hour
> 01_client: actually we've viva for cn
> Ooh, all the best then
> 02_client: thanks..
> 01_client: thanks..bye
> [Server] 01_client has left the chat.
> [Server] 02_client has left the chat.
> ^C
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/7_TCP_Multi_User_Chat_Server$
```

Figure 30: Terminal Output (Client 3)

## RESULT

The program was executed and verified successfully.

# Experiment No. 8

# UDP CONCURRENT TIME SERVER

---

## AIM

Implement a concurrent Time Server application using UDP to execute the program at a remote server. The client sends a time request to the server which sends its system time back. The client then displays the received time value.

## ALGORITHM

### Server Algorithm

1. Start.
2. Create a UDP socket using socket().
3. Initialize the server address structure with the appropriate IP address and port number.
4. Bind the socket to the server address using the bind() system call.
5. Receive the time request packet from a client along with the client's socket address using recvfrom().
6. Fetch the current system time of the server machine.
7. Send the formatted system time string back to the client using sendto() with the client's address details.
8. Close the socket.
9. Stop.

### Client Algorithm

1. Start.
2. Create a UDP socket using socket().

3. Initialize the server address structure with the target server's IP address and port number.
4. Send a time request message to the server using sendto().
5. Wait for and receive the response containing the current system time from the server using recvfrom().
6. Display the received system time value on the client console.
7. Close the socket.
8. Stop.

## PROGRAM

### Server

```
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <ctype.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 1024
int port;
int main (void) {
        char buffer[BUFFER_SIZE];
        struct sockaddr_in server_addr, client_addr;
        socklen_t client_len = sizeof(client_addr);
        printf("Enter port: "); scanf("%d", &port);
        int sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
        if (sock_fd < 0) {
                perror("Socket creation failed");
                return 1;
        }

        server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
        server_addr.sin_port = htons(port);
        server_addr.sin_family = AF_INET;
        if (bind(sock_fd, (struct sockaddr*)&server_addr,
    sizeof(server_addr)) < 0) {
                perror("Server binding failed");
                return 1;
        }
        printf ("\nWaiting for client...\n");
        memset (buffer, 0, sizeof(buffer));
        recvfrom(sock_fd, buffer, BUFFER_SIZE, 0, (struct
    sockaddr*)&client_addr, &client_len);
        printf("\nMessage from client: %s\n", buffer);
        memset(buffer, 0, sizeof(buffer));
        sendto(sock_fd, "Hello from Server", 17, 0, (struct
    sockaddr*)&client_addr, client_len);
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sock_fd, buffer, BUFFER_SIZE, 0, (struct
    sockaddr*)&client_addr, &client_len);
        printf("%s from client\n", buffer);
        time_t raw_time; time(&raw_time);

        printf("\nSending current time: %s", ctime(&raw_time));
        memset(buffer, 0, sizeof(buffer));
        sendto(sock_fd, ctime(&raw_time), BUFFER_SIZE, 0, (struct
    sockaddr*)&client_addr, client_len);
        memset(buffer, 0, sizeof(buffer));
        close(sock_fd);
        printf("\nClient disconnected...\n");
        return 0;
}
```

### Client

```
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <ctype.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 1024

int port;
int main (void) {
        printf("Enter port: "); scanf("%d", &port);
        char buffer[BUFFER_SIZE]; struct sockaddr_in server_addr, client_addr;
        socklen_t client_len = sizeof(client_addr);
        server_addr.sin_addr.s_addr = INADDR_ANY;
        server_addr.sin_port = htons(port);
        server_addr.sin_family = AF_INET;
        int sock_fd = socket(AF_INET, SOCK_DGRAM, 0);

        if (connect(sock_fd, (struct sockaddr*)&server_addr,
    sizeof(server_addr)) < 0) {
                perror("Connection to server failed");
                return 1;
        }
        sendto(sock_fd, "Hello from client", 17, 0, (struct sockaddr*)NULL,
    sizeof(server_addr));
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sock_fd, buffer, sizeof(buffer), 0, (struct sockaddr*)NULL,
    NULL);
        printf("Message from server: %s\n", buffer);
        memset(buffer, 0, sizeof(buffer));

        printf("\nRequesting server for system time...\n");
        sendto(sock_fd, "TIME_REQUEST", 12, 0, (struct sockaddr*)NULL,
    sizeof(server_addr));
        memset(buffer, 0, sizeof(buffer));
        recvfrom(sock_fd, buffer, sizeof(buffer), 0, (struct sockaddr*)NULL,
    NULL);
        printf("\nResponse from server: %s\n", buffer);
        memset(buffer, 0, sizeof(buffer));
        close(sock_fd);
        return 0;
}
```

## OUTPUT

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/8_UDP_Time_Server$ ./server
Enter port: 8888

Waiting for client...

Message from client: Hello from client
TIME_REQUEST from client

Sending current time: Tue Oct 13 18:42:45 2026

Client disconnected...
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/8_UDP_Time_Server$
```

Figure 31: Terminal Output (Server)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/8_UDP_Time_Server$ ./client
Enter port: 8888
Message from server: Hello from Server

Requesting server for system time...

Response from server: Tue Oct 13 18:42:45 2026

student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/8_UDP_Time_Server$
```

Figure 32: Terminal Output (Client)

## RESULT

The program was executed and verified successfully.

# Experiment No. 9

# CONCURRENT FILE SERVER

---

## AIM

Develop a concurrent file server that will provide the file requested by the client if it exists. If not, the server sends an appropriate message to the client. The server should also send its process ID (PID) to clients for displaying along with the file contents or with the message.

## ALGORITHM

### Server Algorithm

1. Start.
2. Create a socket (socket()) for IPv4 and TCP communication.
3. Bind the socket (bind()) to a specific IP address and port number.
4. Listen (listen()) on the socket for incoming client connection requests.
5. Repeat indefinitely (Main Loop):
   (a) Accept (accept()) an incoming connection from a client, generating a new connected socket descriptor.
   (b) Fork (fork()) a new child process:
   - If fork() = 0 (Child Process):
     i. Close the listening socket descriptor in the child (not needed for communication).
     ii. Receive the requested filename string sent by the client.
     iii. Obtain the current process ID using getpid().
     iv. Check if the requested file exists in the server directory (e.g., using access() or fopen()).
     v. If the file exists:
     – Send a success status or file existence confirmation along with the server PID.

- Read the file contents block by block and send them to the client.
- Close the file.

vi. If the file does not exist:

- Send an appropriate error message ("File not found") along with the server PID.

vii. Close the connected client socket and exit the child process.

- If fork() > 0 (Parent Process):

i. Close the connected socket descriptor in the parent.

ii. Loop back to accept the next client connection (concurrent handling).

6. Close the server socket.
7. Stop.

**Client Algorithm**

1. Start.
2. Create a socket (socket()) for TCP communication.
3. Connect (connect()) to the server using the server's IP address and port number.
4. Prompt the user to enter the name of the file they want to request.
5. Send the filename string to the server over the socket connection.
6. Receive data from the server:
   - Receive and display the server's Process ID (PID).
   - Receive and display the status message or the contents of the requested file stream.
7. Close the socket connection.
8. Stop.

**PROGRAM**

**Server**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUFFER_SIZE 256
int port;
void handle_client(int client_sock) {
    char buffer[BUFFER_SIZE];
    char filename[BUFFER_SIZE];
    pid_t pid = getpid();
    int n = read(client_sock, filename, sizeof(filename) - 1);
    if (n <= 0) {
        close(client_sock);
        exit(0);
    }
    printf("\nIncoming request for file: \"%s\"\n", filename);
    filename[n] = '\0';
    filename[strcspn(filename, "\r\n")] = 0;
    FILE *f = fopen(filename, "r");
    if (f == NULL) {
        snprintf(buffer, sizeof(buffer), "[Server PID: %d] Error: File
    '%.100s' not found.\n", pid, filename);
        write(client_sock, buffer, strlen(buffer));
        printf("\nError: File \"%.100s\" not found.\n", filename);
    } else {
        snprintf(buffer, sizeof(buffer), "[Server PID: %d] File
    Contents:\n----------------------------------------\n", pid);
        write(client_sock, buffer, strlen(buffer));
        size_t bytes_read;
        while ((bytes_read = fread(buffer, 1, sizeof(buffer), f)) > 0) {
            write(client_sock, buffer, bytes_read);
        }
        fclose(f);
        printf("\nFile: \"%s\" sent to client.\n", filename);
    }
    close(client_sock);
    exit(0);
}
```

```
int main() {
    int server_sock, client_sock;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;

    server_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (server_sock < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Enter port: "); scanf("%d", &port);
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(port);
    if (bind(server_sock, (struct sockaddr*)&server_addr,
   sizeof(server_addr)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_sock, 10) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
    printf("File Server started on port %d...\n", port);
    while (1) {
        addr_size = sizeof(client_addr);
        client_sock = accept(server_sock, (struct sockaddr*)&client_addr,
   &addr_size);
        if (client_sock < 0) {
            perror("Accept failed");
            continue;
        }
        if (fork() == 0) {
            close(server_sock);
            handle_client(client_sock);
        } else {
            close(client_sock);
        }
    }

    close(server_sock);
```

```
    printf("\nClient disconnected...\n");
    return 0;
}
```

### Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 256
int port ;
int main() {
    int sock;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE];
    char filename[BUFFER_SIZE];

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }
    printf("Enter port: "); scanf("%d", &port);
    getchar();
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
    printf("Enter filename to request: ");
    fgets(filename, sizeof(filename), stdin);
    filename[strcspn(filename, "\n")] = 0;
    write(sock, filename, strlen(filename));
```

```
    int n;
    while ((n = read(sock, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[n] = '\0';
        printf("%s", buffer);
    }
    close(sock);
    return 0;
}
```

## OUTPUT

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$ ls
client  Q9_Client.c  Q9_Server.c  sample1.txt  sample2.txt  server
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$ ./server
Enter port: 5173
File Server started on port 5173...

Incoming request for file: "sample1.txt"

File: "sample1.txt" sent to client.

Incoming request for file: "nothing.txt"

Error: File "nothing.txt" not found.
^C
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$
```

Figure 33: Terminal Output (Server)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$ ./client
Enter port: 5173
Enter filename to request: sample1.txt
[Server PID: 102067] File Contents:
---------------------------------------
This is a sample text file.

student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$
```

Figure 34: Terminal Output (Client: file exists)

```
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$ ./client
Enter port: 5173
Enter filename to request: nothing.txt
[Server PID: 102086] Error: File 'nothing.txt' not found.
student@cse-lab:~/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/9_File_Server$
```

Figure 35: Terminal Output (Client: file not found)

## RESULT

The program was executed and verified successfully.

# Experiment No. 10

# PACKET CAPTURE USING RAW SOCKETS

---

### AIM

Develop a packet-capturing application using raw sockets.

### ALGORITHM

1. Start.
2. Create a raw socket using socket() with the address family AF_PACKET, socket type SOCK_RAW and protocol ETH_P_ALL (converted with htons()). This receives all incoming and outgoing Ethernet frames at the link layer. If socket creation fails, print an error message and stop.
3. Allocate a buffer of 65536 bytes in memory to hold the captured packets. If the allocation fails, print an error message, close the socket and stop.
4. Display a message indicating that packet capture has started and that Ctrl+C can be pressed to stop it.
5. Enter an infinite loop to capture packets continuously:
   - Receive a packet into the buffer using recvfrom().
   - If recvfrom() returns an error, print an error message and exit the loop.
   - Otherwise, pass the received packet and its size to the packet processing function.
6. Process each received packet as follows:
   - Treat the first 14 bytes of the buffer as the Ethernet header.
   - Print the Ethernet header details: the destination MAC address (bytes 0–5), the source MAC address (bytes 6–11) and the protocol type (bytes 12–13) in hexadecimal.
   - If the protocol type is 0x0800 (IPv4), treat the data immediately after the Ethernet header as the IP header. Otherwise, skip the IP header details.

- Print the IP header details: the IP version, the protocol number along with its name (TCP for 6, UDP for 17, ICMP for 1, and Other for any other value), the source IP address and the destination IP address.
- Print a separator line after each packet.

7. Return to the capture loop to receive the next packet.
8. When the loop ends, free the buffer and close the raw socket.
9. Stop.

**PROGRAM**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <net/ethernet.h>

#define BUFFER_SIZE 65536
void process_packet(unsigned char* buffer, int size);
void print_ethernet_header(unsigned char* buffer, int size);
void print_ip_header(unsigned char* buffer, int size);
int main() {
    int sock_raw;
    struct sockaddr saddr;
    int saddr_len = sizeof(saddr);

    // Create raw socket using AF_PACKET to capture all Ethernet frames
    sock_raw = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    if (sock_raw < 0) {
        perror("Socket Error. Make sure you are using unshare or have
   capabilities.");
        return 1;
    }
    unsigned char *buffer = (unsigned char *)malloc(BUFFER_SIZE);
    if (!buffer) {
```

```
        perror("Memory allocation failed");
        close(sock_raw);
        return 1;
    }
    printf("Starting packet capture (with MAC parsing)... Press Ctrl+C to
    stop.\n\n");
    while (1) {
        int data_size = recvfrom(sock_raw, buffer, BUFFER_SIZE, 0, &saddr,
    (socklen_t*)&saddr_len);
        if (data_size < 0) {
            perror("Recvfrom error");
            break;
        }
        process_packet(buffer, data_size);
    }
    free(buffer);
    close(sock_raw);
    return 0;
}
void process_packet(unsigned char* buffer, int size) {
    struct ethhdr *eth = (struct ethhdr *)buffer;
    // 1. Print Layer 2 Ethernet Details (MAC Addresses)
    print_ethernet_header(buffer, size);
    // 2. If it's an IP packet, print Layer 3 IP Details
    if (ntohs(eth->h_proto) == ETH_P_IP) {
        print_ip_header(buffer + sizeof(struct ethhdr), size - sizeof(struct
    ethhdr));
    }
    printf("\n----------------------------------------\n");
}
void print_ethernet_header(unsigned char* buffer, int size) {
    struct ethhdr *eth = (struct ethhdr *)buffer;
    printf("--- Ethernet Header ---\n");
    printf("  |- Destination MAC : %.2X:%.2X:%.2X:%.2X:%.2X:%.2X\n",
           eth->h_dest[0], eth->h_dest[1], eth->h_dest[2], eth->h_dest[3],
    eth->h_dest[4], eth->h_dest[5]);
    printf("  |- Source MAC      : %.2X:%.2X:%.2X:%.2X:%.2X:%.2X\n",
           eth->h_source[0], eth->h_source[1], eth->h_source[2],
    eth->h_source[3], eth->h_source[4], eth->h_source[5]);
    printf("  |- Protocol Type   : 0x%.4x\n", ntohs(eth->h_proto));
}
```

```
void print_ip_header(unsigned char* buffer, int size) {
    struct iphdr *iph = (struct iphdr *)buffer;
    struct sockaddr_in source, dest;
    memset(&source, 0, sizeof(source));
    source.sin_addr.s_addr = iph->saddr;

    memset(&dest, 0, sizeof(dest));
    dest.sin_addr.s_addr = iph->daddr;
    printf("--- IP Header ---\n");
    printf("  |- Version         : %d\n", (unsigned int)iph->version);
    printf("  |- Protocol        : %d ", (unsigned int)iph->protocol);
    if (iph->protocol == IPPROTO_TCP) {
        printf("(TCP)\n");
    } else if (iph->protocol == IPPROTO_UDP) {
        printf("(UDP)\n");
    } else if (iph->protocol == IPPROTO_ICMP) {
        printf("(ICMP)\n");
    } else {
        printf("(Other)\n");
    }
    printf("  |- Source IP       : %s\n", inet_ntoa(source.sin_addr));
    printf("  |- Dest IP         : %s\n", inet_ntoa(dest.sin_addr));
}
```

## OUTPUT

```
student@cse-lab:~/cse/lab/Networks-Lab-Sem-5-CS/Lab_Cycle_Solutions/10_Packet_Capture_Tool$ sudo ./capture
Starting packet capture (with MAC parsing)... Press Ctrl+C to stop.

--- Ethernet Header ---
  |- Destination MAC : 00:00:00:00:00:00
  |- Source MAC      : 00:00:00:00:00:00
  |- Protocol Type   : 0x0800
--- IP Header ---
  |- Version         : 4
  |- Protocol        : 6 (TCP)
  |- Source IP       : 127.0.0.1
  |- Dest IP         : 127.0.0.1

----------------------------------------
--- Ethernet Header ---
  |- Destination MAC : 00:00:00:00:00:00
  |- Source MAC      : 00:00:00:00:00:00
  |- Protocol Type   : 0x0800
--- IP Header ---
  |- Version         : 4
  |- Protocol        : 6 (TCP)
  |- Source IP       : 127.0.0.1
  |- Dest IP         : 127.0.0.1

----------------------------------------
--- Ethernet Header ---
  |- Destination MAC : 00:00:00:00:00:00
  |- Source MAC      : 00:00:00:00:00:00
  |- Protocol Type   : 0x0800
--- IP Header ---
  |- Version         : 4
  |- Protocol        : 6 (TCP)
  |- Source IP       : 127.0.0.1
  |- Dest IP         : 127.0.0.1
```

Figure 36: Terminal Output

## RESULT

The program was executed and verified successfully.

# Experiment No. 11

# FAMILIARIZING ROUTER COMMANDS

---

**AIM**

To familiarize with basic Cisco IOS router commands in Cisco Packet Tracer:

(a) Knowing the current mode (user or privileged), switching to privileged mode.

(b) Switching to configuration mode.

(c) Obtaining router information such as type, OS, memory stats, interface details, etc.

(d) Viewing the status of any routing protocols currently configured.

(e) Showing the routing table.

(f) Saving the running configuration.

(g) Viewing the command history.

(h) Viewing the router clock.

(i) Viewing the list of hosts.

(j) Displaying the statistics for all the interfaces (both detailed and brief views).

(k) Knowing the controller type (DTE or DCE).

(l) Configuring serial and Ethernet interfaces: enabling the interface, setting IP address, subnet mask, and clock rate.

**REQUIREMENTS**

- Cisco Packet Tracer
- Cisco 1841 Router
- Serial DCE/DTE and Copper Straight-Through Cables

## THEORY

Cisco routers run the Internetwork Operating System (IOS), which provides a Command Line Interface (CLI) divided into hierarchical command modes for security and administration:

- **User EXEC Mode (`Router>`):** Allows limited examination of router status without modifying configurations.
- **Privileged EXEC Mode (`Router#`):** Provides access to all diagnostic, inspection, and configuration management commands.
- **Global Configuration Mode (`Router(config)#`):** Used to modify system-wide router parameters.
- **Interface Configuration Mode (`Router(config-if)#`):** Used to configure specific router interfaces such as FastEthernet and Serial ports.

## ROUTER COMMANDS AND CONFIGURATION

### (a) Knowing the Current Mode and Switching to Privileged Mode

The prompt `Router>` indicates User EXEC mode, while `Router#` indicates Privileged EXEC mode.

```
Router> enable
Router#
```

### (b) Switching to Configuration Mode

```
Router# configure terminal
Router(config)# exit
Router#
```

### (c) Obtaining Router Information (Type, OS, Memory, Interfaces)

```
Router# show version
```

The output shows a Cisco 1841 router running IOS 12.4(15)T1, with 114688K/16384K bytes of memory, 2 FastEthernet interfaces, 4 low-speed serial interfaces, 191K bytes of NVRAM and 63488K bytes of CompactFlash.

### (d) Viewing the Status of Configured Routing Protocols

```
Router# show ip protocols
```

The router is running RIP (version 2) for the network 172.16.0.0, with updates sent every 30 seconds.

### (e) Showing the Routing Table

```
Router# show ip route
```

### (f) Saving the Running Configuration

```
Router# copy running-config startup-config
! Alternative command with the same effect:
Router# write memory
```

### (g) Viewing the Command History

```
Router# show history
```

### (h) Viewing the Router Clock

```
Router# show clock
```

### (i) Viewing the List of Hosts

```
Router# show hosts
```

### (j) Displaying Statistics for All Interfaces (Detailed and Brief)

```
Router# show interfaces
Router# show interfaces fastethernet 0/0
Router# show ip interface brief
```

### (k) Knowing the Controller Type (DTE or DCE)

```
Router# show controllers serial 0/0/0
```

The output reports `DTE V.35 TX and RX clocks detected`, so Serial0/0/0 is the DTE end of its link and does not need a clock rate.

### (l) Configuring Serial and Ethernet Interfaces

```
Router> enable
Router# configure terminal

! Configuring FastEthernet Interface
Router(config)# interface fastethernet 0/0
Router(config-if)# ip address 172.16.30.1 255.255.255.0
Router(config-if)# no shutdown

! Configuring Serial Interface (DCE end)
Router(config-if)# interface serial 0/0/1
Router(config-if)# ip address 172.16.40.1 255.255.255.0
Router(config-if)# clock rate 64000
Router(config-if)# no shutdown
Router(config-if)# end
```

**OUTPUT**

```
Router>enable
Router#show interfaces fastethernet 0/0
FastEthernet0/0 is up, line protocol is up (connected)
  Hardware is Lance, address is 0090.2b45.6901 (bia 0090.2b45.6901)
  Internet address is 172.16.10.2/24
  MTU 1500 bytes, BW 100000 Kbit, DLY 100 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Full-duplex, 100Mb/s, media type is RJ45
  ARP type: ARPA, ARP Timeout 04:00:00,
  Last input 00:00:08, output 00:00:05, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/75/0 (size/max/drops); Total output drops: 0
  Queueing strategy: fifo
  Output queue :0/40 (size/max)
  5 minute input rate 4 bits/sec, 0 packets/sec
  5 minute output rate 9 bits/sec, 0 packets/sec
     3 packets input, 182 bytes, 0 no buffer
     Received 2 broadcasts, 0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored, 0 abort
     0 input packets with dribble condition detected
     5 packets output, 400 bytes, 0 underruns
     0 output errors, 0 collisions, 1 interface resets
     0 babbles, 0 late collision, 0 deferred

Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface fastethernet 0/0
Router(config-if)#ip address 172.16.30.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#interface serial 0/0/1
Router(config-if)#ip address 172.16.40.1 255.255.255.0
Router(config-if)#clock rate 64000
Router(config-if)#no shutdown
Router(config-if)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#
```

Figure 37: `enable`, `configure terminal`, `exit` and `show version`

```
Router#show controllers serial 0/0/0
Interface Serial0/0/0
Hardware is PowerQUICC MPC860
DTE V.35 TX and RX clocks detected
idb at 0x81081AC4, driver data structure at 0x81084AC0
SCC Registers:
General [GSMR]=0x2:0x00000000, Protocol-specific [PSMR]=0x8
Events [SCCE]=0x0000, Mask [SCCM]=0x0000, Status [SCCS]=0x00
Transmit on Demand [TODR]=0x0, Data Sync [DSR]=0x7E7E
Interrupt Registers:
Config [CICR]=0x00367F80, Pending [CIPR]=0x0000C000
Mask   [CIMR]=0x00200000, In-srv  [CISR]=0x00000000
Command register [CR]=0x580
Port A [PADIR]=0x1030, [PAPAR]=0xFFFF
       [PAODR]=0x0010, [PADAT]=0xCBFF
Port B [PBDIR]=0x09C0F, [PBPAR]=0x0800E
       [PBODR]=0x00000, [PBDAT]=0x3FFFD
Port C [PCDIR]=0x00C, [PCPAR]=0x200
       [PCSO]=0xC20,  [PCDAT]=0xDF2, [PCINT]=0x00F
```

Figure 38: `show ip protocols`

```
Router#show clock
*0:6:15.643 UTC Mon Mar 1 1993
Router#show history
  enable
  configure terminal
  show version
  show ip protocols
  show ip route
  copy running-config startup-config
  show clock
  show history
Router#show hosts
Default Domain is not set
Name/address lookup uses domain service
Name servers are 255.255.255.255

Codes: UN - unknown, EX - expired, OK - OK, ?? - revalidate
       temp - temporary, perm - permanent
       NA - Not Applicable None - Not defined

Host                      Port  Flags      Age Type   Address(es)
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
FastEthernet0/0        172.16.30.1     YES manual up                    up
FastEthernet0/1        unassigned      YES unset  administratively down down
Serial0/0/0            172.16.20.2     YES manual up                    up
Serial0/0/1            172.16.40.1     YES manual up                    up
Serial0/1/0            unassigned      YES unset  administratively down down
Serial0/1/1            unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

Figure 39: `show ip route` and `copy running-config startup-config`

```
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     172.16.0.0/24 is subnetted, 7 subnets
R       172.16.10.0 [120/1] via 172.16.20.1, 00:00:14, Serial0/0/0
C       172.16.20.0 is directly connected, Serial0/0/0
C       172.16.30.0 is directly connected, FastEthernet0/0
C       172.16.40.0 is directly connected, Serial0/0/1
R       172.16.50.0 [120/1] via 172.16.40.2, 00:00:10, Serial0/0/1
R       172.16.60.0 [120/1] via 172.16.20.1, 00:00:14, Serial0/0/0
R       172.16.70.0 [120/1] via 172.16.40.2, 00:00:10, Serial0/0/1

Router#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#
```

Figure 40: `show clock`, `show history`, `show hosts` and `show ip interface brief`

```
Router#show ip protocols
Routing Protocol is "rip"
Sending updates every 30 seconds, next due in 21 seconds
Invalid after 180 seconds, hold down 180, flushed after 240
Outgoing update filter list for all interfaces is not set
Incoming update filter list for all interfaces is not set
Redistributing: rip
Default version control: send version 2, receive 2
  Interface             Send  Recv  Triggered RIP  Key-chain
  FastEthernet0/0       22
  Serial0/0/1           22
  Serial0/0/0           22
Automatic network summarization is not in effect
Maximum path: 4
Routing for Networks:
         172.16.0.0
Passive Interface(s):
Routing Information Sources:
         Gateway         Distance      Last Update
         172.16.20.1           120      00:00:09
         172.16.40.2           120      00:00:05
Distance: (default is 120)
```

Figure 41: `show controllers serial 0/0/0`

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show version
Cisco IOS Software, 1841 Software (C1841-ADVIPSERVICESK9-M), Version 12.4(15)T1, RELEASE SOFTWARE (fc2)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2007 by Cisco Systems, Inc.
Compiled Wed 18-Jul-07 04:52 by pt_team

ROM: System Bootstrap, Version 12.3(8r)T8, RELEASE SOFTWARE (fc1)

System returned to ROM by power-on
System image file is "flash:c1841-advipservicesk9-mz.124-15.T1.bin"


This product contains cryptographic features and is subject to United
States and local country laws governing import, export, transfer and
use. Delivery of Cisco cryptographic products does not imply
third-party authority to import, export, distribute or use encryption.
Importers, exporters, distributors and users are responsible for
compliance with U.S. and local country laws. By using this product you
agree to comply with applicable laws and regulations. If you are unable
to comply with U.S. and local laws, return this product immediately.

A summary of U.S. laws governing Cisco cryptographic products may be found at:
http://www.cisco.com/wwl/export/crypto/tool/stqrg.html

If you require further assistance please contact us by sending email to
export@cisco.com.

Cisco 1841 (revision 5.0) with 114688K/16384K bytes of memory.
Processor board ID FTX0947Z18E
M860 processor: part number 0, mask 49
2 FastEthernet/IEEE 802.3 interface(s)
4 Low-speed serial(sync/async) network interface(s)
191K bytes of NVRAM.
63488K bytes of ATA CompactFlash (Read/Write)

Configuration register is 0x2102
```

Figure 42: `show interfaces fastethernet 0/0` and configuration of FastEthernet 0/0 and Serial 0/0/1

**RESULT**

The basic Cisco IOS router commands and interface configuration steps were executed and verified successfully.

# Experiment No. 12

# IMPLEMENT STATIC ROUTING

---

### AIM

To set up static routing for the given network topology, display the routing table, and verify end-to-end connectivity using the `ping` command.

### REQUIREMENTS

- Cisco Packet Tracer
- One Cisco `2621XM` router and four Cisco `1841` routers
- One PC
- Copper cross-over cables
- Serial DCE/DTE cables

### THEORY

Static routing is a form of routing that occurs when a router uses a manually configured routing entry, rather than information from dynamic routing traffic. Network administrators manually configure static routes in a router's routing table using the global configuration command:

```
ip route <destination-network> <subnet-mask> <next-hop-ip-address>
```

Routes configured statically are denoted by the code `S` in the routing table and have a default Administrative Distance (AD) of **1**.

## NETWORK TOPOLOGY

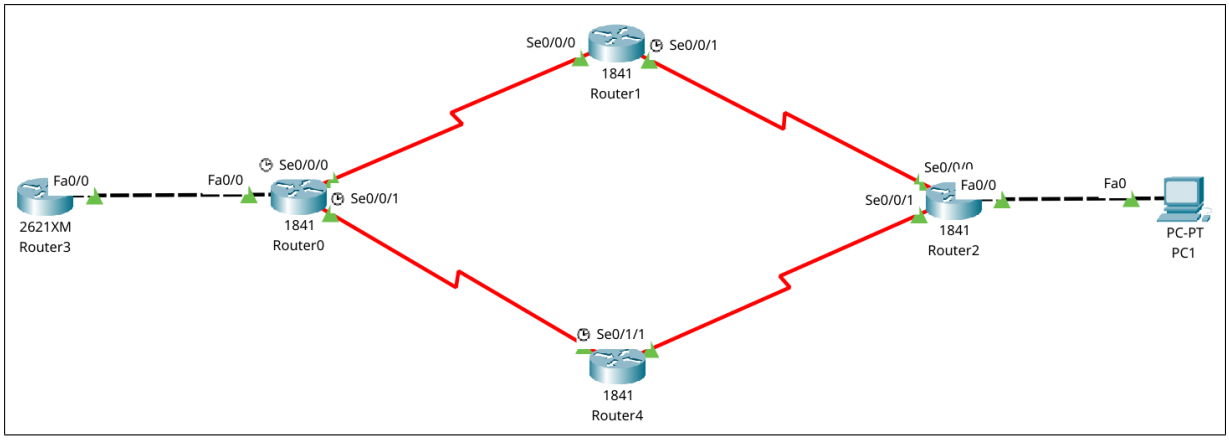



Figure 43: Static Routing Network Topology

## IP ADDRESS ALLOCATION

| Device | Interface | IP Address | Subnet Mask |
|---|---|---|---|
| Router3 (2621XM) | FastEthernet0/0 | 172.16.10.1 | 255.255.255.0 |
| Router0 (1841) | FastEthernet0/0 | 172.16.10.2 | 255.255.255.0 |
| Router0 (1841) | Serial0/0/0 (DCE) | 172.16.20.1 | 255.255.255.0 |
| Router0 (1841) | Serial0/0/1 (DCE) | 172.16.60.1 | 255.255.255.0 |
| Router1 (1841) | Serial0/0/0 (DTE) | 172.16.20.2 | 255.255.255.0 |
| Router1 (1841) | Serial0/0/1 (DCE) | 172.16.40.1 | 255.255.255.0 |
| Router2 (1841) | Serial0/0/0 (DTE) | 172.16.40.2 | 255.255.255.0 |
| Router2 (1841) | Serial0/0/1 (DTE) | 172.16.70.2 | 255.255.255.0 |
| Router2 (1841) | FastEthernet0/0 | 172.16.30.1 | 255.255.255.0 |
| Router4 (1841) | Serial0/1/0 (DTE) | 172.16.60.2 | 255.255.255.0 |
| Router4 (1841) | Serial0/1/1 (DCE) | 172.16.70.1 | 255.255.255.0 |
| PC1 (PC-PT) | FastEthernet0 | 172.16.30.10 | 255.255.255.0 |

Table 3: IP Address Allocation

## NETWORK ADDRESSES

| Network | Network Address |
|---|---|
| Router3 – Router0 | 172.16.10.0/24 |
| Router0 – Router1 | 172.16.20.0/24 |
| Router1 – Router2 | 172.16.40.0/24 |
| Router0 – Router4 | 172.16.60.0/24 |
| Router4 – Router2 | 172.16.70.0/24 |
| Router2 – PC1 | 172.16.30.0/24 |

Table 4: Network Addresses

## CONNECTIVITY

| Device | Interface | Connected Device | Interface |
|---|---|---|---|
| Router3 | FastEthernet0/0 | Router0 | FastEthernet0/0 |
| Router0 | Serial0/0/0 (DCE) | Router1 | Serial0/0/0 (DTE) |
| Router0 | Serial0/0/1 (DCE) | Router4 | Serial0/1/0 (DTE) |
| Router1 | Serial0/0/1 (DCE) | Router2 | Serial0/0/0 (DTE) |
| Router4 | Serial0/1/1 (DCE) | Router2 | Serial0/0/1 (DTE) |
| Router2 | FastEthernet0/0 | PC1 | FastEthernet0 |

Table 5: Network Connectivity

## PROCEDURE

1. Connect the five routers (`Router3`, `Router0`, `Router1`, `Router2`, `Router4`) and `PC1` according to the given topology.
2. Assign IP addresses and subnet masks to all router interfaces.
3. Configure the DCE serial interfaces with a clock rate of `64000`.
4. Enable all required router interfaces using `no shutdown`.
5. Configure the IP address (`172.16.30.10`), subnet mask (`255.255.255.0`), and default gateway (`172.16.30.1`) on `PC1`.
6. Verify interface statuses using `show ip interface brief`.
7. Configure static routes on the routers to reach the remote networks using the `ip route` command.

8. Display the routing table on the router using `show ip route`.
9. Verify end-to-end connectivity across the network using the `ping` command.

## CONFIGURATION

### Interface Configuration

### Router3 (2621XM)

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.10.1 255.255.255.0
no shutdown
end
```

### Router0 (1841)

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.10.2 255.255.255.0
no shutdown
exit
interface serial0/0/0
ip address 172.16.20.1 255.255.255.0
clock rate 64000
no shutdown
exit
interface serial0/0/1
ip address 172.16.60.1 255.255.255.0
clock rate 64000
no shutdown
end
```

### Router1 (1841)

```
enable
configure terminal
interface serial0/0/0
ip address 172.16.20.2 255.255.255.0
no shutdown
exit
```

```
interface serial0/0/1
ip address 172.16.40.1 255.255.255.0
clock rate 64000
no shutdown
end
```

### Router2 (1841)

```
enable
configure terminal
interface serial0/0/0
ip address 172.16.40.2 255.255.255.0
no shutdown
exit
interface serial0/0/1
ip address 172.16.70.2 255.255.255.0
no shutdown
exit
interface fastethernet0/0
ip address 172.16.30.1 255.255.255.0
no shutdown
end
```

### Router4 (1841)

```
enable
configure terminal
interface serial0/1/0
ip address 172.16.60.2 255.255.255.0
no shutdown
exit
interface serial0/1/1
ip address 172.16.70.1 255.255.255.0
clock rate 64000
no shutdown
end
```

## Static Route Configuration

### Router3 (2621XM)

```
enable
configure terminal
```

```
ip route 172.16.20.0 255.255.255.0 172.16.10.2
ip route 172.16.30.0 255.255.255.0 172.16.10.2
ip route 172.16.40.0 255.255.255.0 172.16.10.2
end
```

### Router0 (1841)

```
enable
configure terminal
ip route 172.16.30.0 255.255.255.0 172.16.20.2
ip route 172.16.40.0 255.255.255.0 172.16.20.2
ip route 172.16.70.0 255.255.255.0 172.16.60.2
end
```

### Router1 (1841)

```
enable
configure terminal
ip route 172.16.10.0 255.255.255.0 172.16.20.1
ip route 172.16.30.0 255.255.255.0 172.16.40.2
end
```

### Router4 (1841)

```
enable
configure terminal
ip route 172.16.10.0 255.255.255.0 172.16.60.1
ip route 172.16.30.0 255.255.255.0 172.16.70.2
end
```

### Router2 (1841)

```
enable
configure terminal
ip route 172.16.10.0 255.255.255.0 172.16.40.1
ip route 172.16.20.0 255.255.255.0 172.16.40.1
ip route 172.16.60.0 255.255.255.0 172.16.70.1
end
```

## VERIFICATION

### Routing Table Verification

On `Router3`, display the routing table using:

```
show ip route
```

Static routes are indicated by the letter `S` in the routing table:

- `S 172.16.20.0 [1/0] via 172.16.10.2`
- `S 172.16.30.0 [1/0] via 172.16.10.2`
- `S 172.16.40.0 [1/0] via 172.16.10.2`

### Connectivity Verification

```
! From PC1:
C:\> ping 172.16.30.1
C:\> ping 172.16.10.1
C:\> ping 172.16.40.2
```

### OUTPUT

```
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     172.16.0.0/24 is subnetted, 4 subnets
C       172.16.10.0 is directly connected, FastEthernet0/0
S       172.16.20.0 [1/0] via 172.16.10.2
S       172.16.30.0 [1/0] via 172.16.10.2
S       172.16.40.0 [1/0] via 172.16.10.2

Router#
```

Figure 44: Routing Table Output (show ip route)

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 172.16.30.1

Pinging 172.16.30.1 with 32 bytes of data:

Reply from 172.16.30.1: bytes=32 time<1ms TTL=255
Reply from 172.16.30.1: bytes=32 time<1ms TTL=255
Reply from 172.16.30.1: bytes=32 time<1ms TTL=255
Reply from 172.16.30.1: bytes=32 time<1ms TTL=255

Ping statistics for 172.16.30.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 172.16.10.1

Pinging 172.16.10.1 with 32 bytes of data:

Reply from 172.16.10.1: bytes=32 time=65ms TTL=253
Reply from 172.16.10.1: bytes=32 time=43ms TTL=253
Reply from 172.16.10.1: bytes=32 time=31ms TTL=253
Reply from 172.16.10.1: bytes=32 time=35ms TTL=253

Ping statistics for 172.16.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 31ms, Maximum = 65ms, Average = 43ms

C:\>ping 172.16.40.2

Pinging 172.16.40.2 with 32 bytes of data:

Reply from 172.16.40.2: bytes=32 time=38ms TTL=254
Reply from 172.16.40.2: bytes=32 time=33ms TTL=254
Reply from 172.16.40.2: bytes=32 time=24ms TTL=254
Reply from 172.16.40.2: bytes=32 time=18ms TTL=254

Ping statistics for 172.16.40.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 18ms, Maximum = 38ms, Average = 28ms

C:\>
```

Figure 45: Connectivity Verification using Ping

**RESULT**

Static routing was successfully configured on all routers, the routing tables were displayed, and end-to-end connectivity between all networks was verified using `ping`.

# Experiment No. 13

# IMPLEMENT RIPv2 ROUTING

---

### AIM

To configure RIPv2 (Routing Information Protocol Version 2) on the routers and verify connectivity using the routing table and `ping` commands.

### REQUIREMENTS

- Cisco Packet Tracer
- One Cisco `2621XM` router and four Cisco `1841` routers
- Two PCs
- Copper cross-over cables
- Serial DCE/DTE cables

### THEORY

RIPv2 is a dynamic distance-vector routing protocol used by routers to exchange routing information and automatically learn routes to remote networks. RIP uses **hop count** as its routing metric (maximum 15 hops).

RIPv2 is enabled using the `version 2` command. The `no auto-summary` command disables automatic classful route summarization. The `network` command specifies the classful network addresses on which RIP operates.

Routes learned through RIP are identified by the letter `R` in the routing table. RIP has an Administrative Distance of **120**.

## NETWORK TOPOLOGY

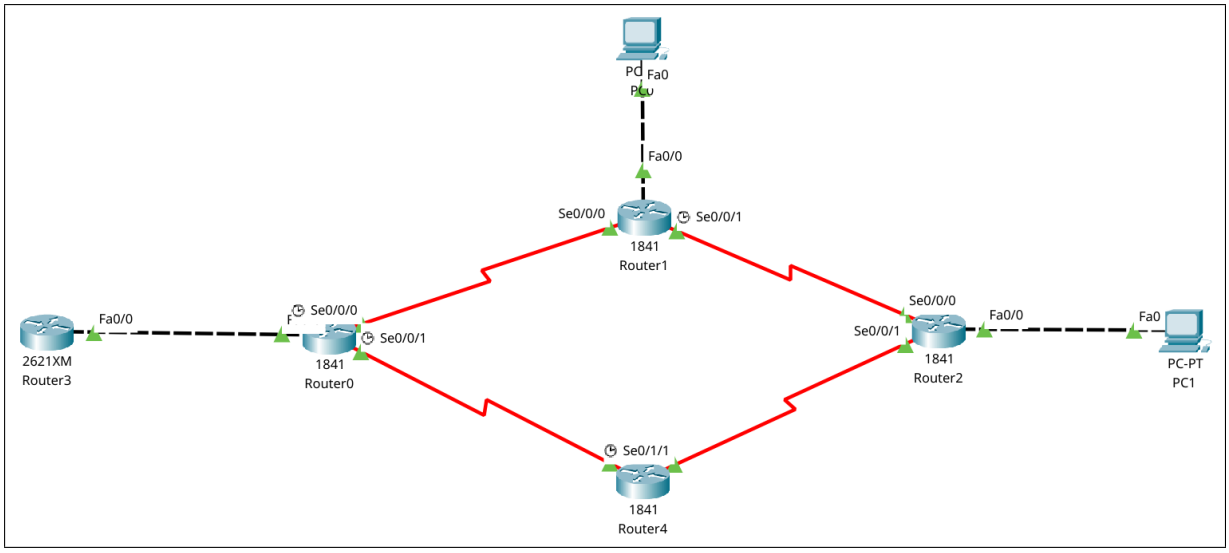



Figure 46: RIPv2 Network Topology

## IP ADDRESS ALLOCATION

| Device | Interface | IP Address | Subnet Mask |
|---|---|---|---|
| Router3 (2621XM) | FastEthernet0/0 | 172.16.10.1 | 255.255.255.0 |
| Router0 (1841) | FastEthernet0/0 | 172.16.10.2 | 255.255.255.0 |
| Router0 (1841) | Serial0/0/0 (DCE) | 172.16.20.1 | 255.255.255.0 |
| Router0 (1841) | Serial0/0/1 (DCE) | 172.16.60.1 | 255.255.255.0 |
| Router1 (1841) | Serial0/0/0 (DTE) | 172.16.20.2 | 255.255.255.0 |
| Router1 (1841) | FastEthernet0/0 | 172.16.30.1 | 255.255.255.0 |
| Router1 (1841) | Serial0/0/1 (DCE) | 172.16.40.1 | 255.255.255.0 |
| Router2 (1841) | Serial0/0/0 (DTE) | 172.16.40.2 | 255.255.255.0 |
| Router2 (1841) | Serial0/0/1 (DTE) | 172.16.70.2 | 255.255.255.0 |
| Router2 (1841) | FastEthernet0/0 | 172.16.50.1 | 255.255.255.0 |
| Router4 (1841) | Serial0/1/0 (DTE) | 172.16.60.2 | 255.255.255.0 |
| Router4 (1841) | Serial0/1/1 (DCE) | 172.16.70.1 | 255.255.255.0 |
| PC0 (PC-PT) | FastEthernet0 | 172.16.30.2 | 255.255.255.0 |
| PC1 (PC-PT) | FastEthernet0 | 172.16.50.2 | 255.255.255.0 |

Table 6: IP Address Allocation

## NETWORK ADDRESSES

| Network | Network Address |
|---|---|
| Router3 – Router0 | 172.16.10.0/24 |
| Router0 – Router1 | 172.16.20.0/24 |
| Router1 – PC0 | 172.16.30.0/24 |
| Router1 – Router2 | 172.16.40.0/24 |
| Router2 – PC1 | 172.16.50.0/24 |
| Router0 – Router4 | 172.16.60.0/24 |
| Router4 – Router2 | 172.16.70.0/24 |

Table 7: Network Addresses

## CONNECTIVITY

| Device | Interface | Connected Device | Interface |
|---|---|---|---|
| Router3 | FastEthernet0/0 | Router0 | FastEthernet0/0 |
| Router0 | Serial0/0/0 (DCE) | Router1 | Serial0/0/0 (DTE) |
| Router0 | Serial0/0/1 (DCE) | Router4 | Serial0/1/0 (DTE) |
| Router1 | FastEthernet0/0 | PC0 | FastEthernet0 |
| Router1 | Serial0/0/1 (DCE) | Router2 | Serial0/0/0 (DTE) |
| Router4 | Serial0/1/1 (DCE) | Router2 | Serial0/0/1 (DTE) |
| Router2 | FastEthernet0/0 | PC1 | FastEthernet0 |

Table 8: Network Connectivity

## PROCEDURE

1. Connect the five routers (`Router3`, `Router0`, `Router1`, `Router2`, `Router4`) and two PCs (`PC0`, `PC1`) according to the given topology.
2. Assign IP addresses and subnet masks to all router interfaces.
3. Configure the DCE serial interfaces with a clock rate of `64000`.
4. Enable all required router interfaces using `no shutdown`.
5. Configure the IP addresses, subnet masks, and default gateways on `PC0` and `PC1`.
6. Verify the interface status using `show ip interface brief`.
7. Configure RIPv2 on all routers using `router rip`, `version 2`, `no auto-summary`, and `network` statements.

8. Verify the RIP configuration using `show ip protocols`.
9. Display the routing table using `show ip route`.
10. Verify end-to-end connectivity using the `ping` command.

## CONFIGURATION

### Interface Configuration

### Router3 (2621XM)

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.10.1 255.255.255.0
no shutdown
end
```

### Router0 (1841)

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.10.2 255.255.255.0
no shutdown
exit
interface serial0/0/0
ip address 172.16.20.1 255.255.255.0
clock rate 64000
no shutdown
exit
interface serial0/0/1
ip address 172.16.60.1 255.255.255.0
clock rate 64000
no shutdown
end
```

### Router1 (1841)

```
enable
configure terminal
interface serial0/0/0
ip address 172.16.20.2 255.255.255.0
```

```
no shutdown
exit
interface fastethernet0/0
ip address 172.16.30.1 255.255.255.0
no shutdown
exit
interface serial0/0/1
ip address 172.16.40.1 255.255.255.0
clock rate 64000
no shutdown
end
```

**Router2 (1841)**

```
enable
configure terminal
interface serial0/0/0
ip address 172.16.40.2 255.255.255.0
no shutdown
exit
interface serial0/0/1
ip address 172.16.70.2 255.255.255.0
no shutdown
exit
interface fastethernet0/0
ip address 172.16.50.1 255.255.255.0
no shutdown
end
```

**Router4 (1841)**

```
enable
configure terminal
interface serial0/1/0
ip address 172.16.60.2 255.255.255.0
no shutdown
exit
interface serial0/1/1
ip address 172.16.70.1 255.255.255.0
clock rate 64000
no shutdown
end
```

### PC Configuration

- **PC0:** IP Address: `172.16.30.2`, Subnet Mask: `255.255.255.0`, Default Gateway: `172.16.30.1`
- **PC1:** IP Address: `172.16.50.2`, Subnet Mask: `255.255.255.0`, Default Gateway: `172.16.50.1`

### RIPv2 Configuration

#### Router3 (2621XM)

```
enable
configure terminal
router rip
version 2
no auto-summary
network 172.16.10.0
end
```

#### Router0 (1841)

```
enable
configure terminal
router rip
version 2
no auto-summary
network 172.16.10.0
network 172.16.20.0
network 172.16.60.0
end
```

#### Router1 (1841)

```
enable
configure terminal
router rip
version 2
no auto-summary
network 172.16.20.0
network 172.16.30.0
network 172.16.40.0
end
```

#### Router2 (1841)

```
enable
configure terminal
router rip
version 2
no auto-summary
network 172.16.40.0
network 172.16.50.0
network 172.16.70.0
end
```

### Router4 (1841)

```
enable
configure terminal
router rip
version 2
no auto-summary
network 172.16.60.0
network 172.16.70.0
end
```

## VERIFICATION

### RIP Verification

On `Router3`, verify the RIP configuration and routing table using:

```
show ip protocols
show ip route
```

RIP-learned routes are indicated by the letter `R` in the routing table:

- `R 172.16.20.0 [120/1] via 172.16.10.2`
- `R 172.16.30.0 [120/2] via 172.16.10.2`
- `R 172.16.40.0 [120/2] via 172.16.10.2`
- `R 172.16.50.0 [120/3] via 172.16.10.2`
- `R 172.16.60.0 [120/1] via 172.16.10.2`
- `R 172.16.70.0 [120/2] via 172.16.10.2`

### Connectivity Verification

```
! From PC0:
C:\> ping 172.16.10.2
C:\> ping 172.16.60.1
```

## OUTPUT

```
Router>enable
Router#show ip protocols
Routing Protocol is "rip"
Sending updates every 30 seconds, next due in 8 seconds
Invalid after 180 seconds, hold down 180, flushed after 240
Outgoing update filter list for all interfaces is not set
Incoming update filter list for all interfaces is not set
Redistributing: rip
Default version control: send version 2, receive 2
  Interface             Send  Recv  Triggered RIP  Key-chain
  FastEthernet0/0       22
Automatic network summarization is not in effect
Maximum path: 4
Routing for Networks:
         172.16.0.0
Passive Interface(s):
Routing Information Sources:
         Gateway         Distance      Last Update
         172.16.10.2           120      00:00:19
Distance: (default is 120)
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     172.16.0.0/24 is subnetted, 7 subnets
C       172.16.10.0 is directly connected, FastEthernet0/0
R       172.16.20.0 [120/1] via 172.16.10.2, 00:00:26, FastEthernet0/0
R       172.16.30.0 [120/2] via 172.16.10.2, 00:00:26, FastEthernet0/0
R       172.16.40.0 [120/2] via 172.16.10.2, 00:00:26, FastEthernet0/0
R       172.16.50.0 [120/3] via 172.16.10.2, 00:00:26, FastEthernet0/0
R       172.16.60.0 [120/1] via 172.16.10.2, 00:00:26, FastEthernet0/0
R       172.16.70.0 [120/2] via 172.16.10.2, 00:00:26, FastEthernet0/0

Router#
```

Figure 47: RIP Protocol and Routing Table Output (show ip protocols & show ip route)

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 172.16.10.2

Pinging 172.16.10.2 with 32 bytes of data:

Reply from 172.16.10.2: bytes=32 time=29ms TTL=254
Reply from 172.16.10.2: bytes=32 time=1ms TTL=254
Reply from 172.16.10.2: bytes=32 time=26ms TTL=254
Reply from 172.16.10.2: bytes=32 time=26ms TTL=254

Ping statistics for 172.16.10.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 29ms, Average = 20ms

C:\>ping 172.16.60.1

Pinging 172.16.60.1 with 32 bytes of data:

Reply from 172.16.60.1: bytes=32 time=34ms TTL=254
Reply from 172.16.60.1: bytes=32 time=31ms TTL=254
Reply from 172.16.60.1: bytes=32 time=26ms TTL=254
Reply from 172.16.60.1: bytes=32 time=26ms TTL=254

Ping statistics for 172.16.60.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 26ms, Maximum = 34ms, Average = 29ms

C:\>
```

Figure 48: Connectivity Verification using Ping

**RESULT**

RIPv2 was successfully configured on all routers. The routers dynamically learned the remote networks through RIPv2, and connectivity between the networks was successfully verified using the routing table and `ping` commands.

# Experiment No. 14

# IMPLEMENT OSPF ROUTING

---

### AIM

To implement OSPF (Open Shortest Path First) routing for the given network topology, display the routing table, and verify connectivity using the `ping` command.

### REQUIREMENTS

- Cisco Packet Tracer
- One Cisco `2621XM` router and four Cisco `1841` routers
- One PC
- Copper cross-over cables
- Serial DCE/DTE cables

### THEORY

OSPF (Open Shortest Path First) is an interior gateway, link-state dynamic routing protocol that uses Dijkstra's Shortest Path First (SPF) algorithm to compute the shortest path tree to all known destinations. OSPF uses **cost** (inversely proportional to interface bandwidth) as its routing metric.

OSPF is enabled using the `router ospf <process-id>` command, and participating interfaces are advertised using:

```
network <network-address> <wildcard-mask> area <area-id>
```

Routes learned via OSPF are identified by the letter `O` in the routing table and have an Administrative Distance of **110**.

## NETWORK TOPOLOGY

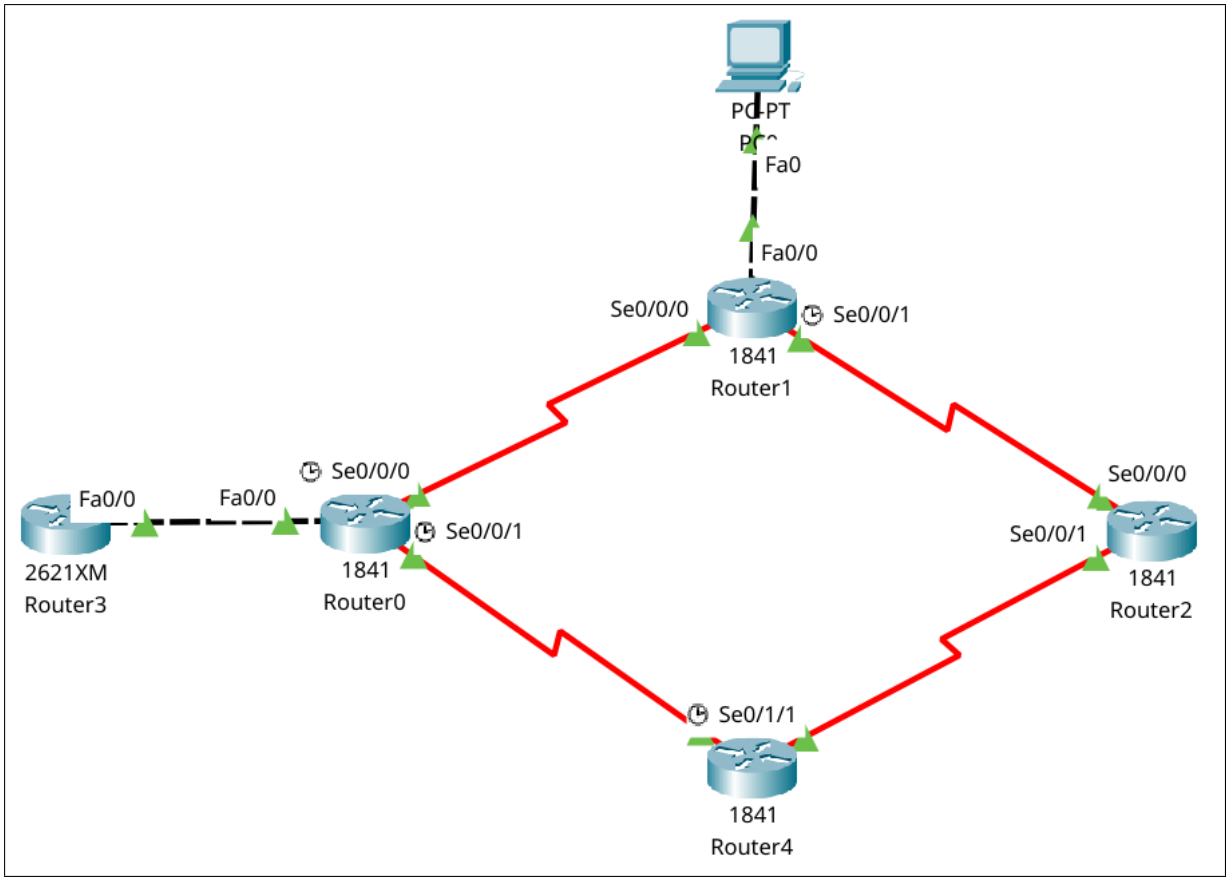



Figure 49: OSPF Network Topology

## IP ADDRESS ALLOCATION

| Device | Interface | IP Address | Subnet Mask |
|---|---|---|---|
| Router3 (2621XM) | FastEthernet0/0 | 172.16.10.1 | 255.255.255.0 |
| Router0 (1841) | FastEthernet0/0 | 172.16.10.2 | 255.255.255.0 |
| Router0 (1841) | Serial0/0/0 (DCE) | 172.16.20.1 | 255.255.255.0 |
| Router0 (1841) | Serial0/0/1 (DCE) | 172.16.60.1 | 255.255.255.0 |
| Router1 (1841) | Serial0/0/0 (DTE) | 172.16.20.2 | 255.255.255.0 |
| Router1 (1841) | FastEthernet0/0 | 172.16.30.1 | 255.255.255.0 |
| Router1 (1841) | Serial0/0/1 (DCE) | 172.16.40.1 | 255.255.255.0 |
| Router2 (1841) | Serial0/0/0 (DTE) | 172.16.40.2 | 255.255.255.0 |
| Router2 (1841) | Serial0/0/1 (DTE) | 172.16.70.2 | 255.255.255.0 |
| Router4 (1841) | Serial0/1/0 (DTE) | 172.16.60.2 | 255.255.255.0 |
| Router4 (1841) | Serial0/1/1 (DCE) | 172.16.70.1 | 255.255.255.0 |
| PC0 (PC-PT) | FastEthernet0 | 172.16.30.10 | 255.255.255.0 |

Table 9: IP Address Allocation

## NETWORK ADDRESSES

| Network | Network Address |
|---|---|
| Router3 – Router0 | 172.16.10.0/24 |
| Router0 – Router1 | 172.16.20.0/24 |
| Router1 – PC0 | 172.16.30.0/24 |
| Router1 – Router2 | 172.16.40.0/24 |
| Router0 – Router4 | 172.16.60.0/24 |
| Router4 – Router2 | 172.16.70.0/24 |

Table 10: Network Addresses

## PROCEDURE

1. Connect the five routers (`Router3`, `Router0`, `Router1`, `Router2`, `Router4`) and `PC0` according to the given topology.
2. Assign IP addresses and subnet masks to all router interfaces and enable them using `no shutdown`.
3. Configure the DCE serial interfaces with a clock rate of `64000`.
4. Configure the IP address (`172.16.30.10`), subnet mask (`255.255.255.0`), and de-

fault gateway (`172.16.30.1`) on `PC0`.

5. Configure single-area OSPF (Process ID `1`, Area `0`) on all five routers using the respective network addresses and wildcard masks (`0.0.0.255`).
6. Verify OSPF neighbor adjacencies using `show ip ospf neighbor`.
7. Display the routing table using `show ip route` and observe the OSPF routes (`O`) and calculated path costs.
8. Verify end-to-end connectivity across the subnets using the `ping` command.

## CONFIGURATION

### OSPF Configuration

#### Router3 (2621XM)

```
enable
configure terminal
router ospf 1
router-id 1.1.1.1
network 172.16.10.0 0.0.0.255 area 0
end
```

#### Router0 (1841)

```
enable
configure terminal
router ospf 1
router-id 2.2.2.2
network 172.16.10.0 0.0.0.255 area 0
network 172.16.20.0 0.0.0.255 area 0
network 172.16.60.0 0.0.0.255 area 0
end
```

#### Router1 (1841)

```
enable
configure terminal
router ospf 1
router-id 3.3.3.3
network 172.16.20.0 0.0.0.255 area 0
network 172.16.30.0 0.0.0.255 area 0
network 172.16.40.0 0.0.0.255 area 0
```

```
end
```

### Router2 (1841)

```
enable
configure terminal
router ospf 1
router-id 4.4.4.4
network 172.16.40.0 0.0.0.255 area 0
network 172.16.70.0 0.0.0.255 area 0
end
```

### Router4 (1841)

```
enable
configure terminal
router ospf 1
router-id 5.5.5.5
network 172.16.60.0 0.0.0.255 area 0
network 172.16.70.0 0.0.0.255 area 0
end
```

## VERIFICATION

### OSPF Verification

On `Router3`, verify the OSPF neighbor table and routing table using:

```
show ip ospf neighbor
show ip route
```

OSPF-learned routes are indicated by the letter `O` in the routing table:

- `O 172.16.20.0 [110/65] via 172.16.10.2`
- `O 172.16.30.0 [110/66] via 172.16.10.2`
- `O 172.16.40.0 [110/129] via 172.16.10.2`
- `O 172.16.60.0 [110/65] via 172.16.10.2`
- `O 172.16.70.0 [110/129] via 172.16.10.2`

### Connectivity Verification

```
! From PC0:
C:\> ping 172.16.70.0
C:\> ping 172.16.30.0
```

### OUTPUT

```
Router>enable
Router#show ip ospf neighbor


Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   FULL/DR         00:00:38    172.16.10.2     FastEthernet0/0
Router#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     172.16.0.0/24 is subnetted, 6 subnets
C       172.16.10.0 is directly connected, FastEthernet0/0
O       172.16.20.0 [110/65] via 172.16.10.2, 00:09:58, FastEthernet0/0
O       172.16.30.0 [110/66] via 172.16.10.2, 00:09:58, FastEthernet0/0
O       172.16.40.0 [110/129] via 172.16.10.2, 00:09:58, FastEthernet0/0
O       172.16.60.0 [110/65] via 172.16.10.2, 00:09:58, FastEthernet0/0
O       172.16.70.0 [110/129] via 172.16.10.2, 00:09:58, FastEthernet0/0

Router#
```

Figure 50: OSPF Neighbor and Routing Table Output (show ip ospf neighbor & show ip route)

```
C:\>ping 172.16.70.0

Pinging 172.16.70.0 with 32 bytes of data:

Reply from 172.16.40.2: bytes=32 time=23ms TTL=254
Reply from 172.16.40.2: bytes=32 time=18ms TTL=254
Reply from 172.16.40.2: bytes=32 time=18ms TTL=254
Reply from 172.16.40.2: bytes=32 time=22ms TTL=254

Ping statistics for 172.16.70.0:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 18ms, Maximum = 23ms, Average = 20ms

C:\>ping 172.16.30.0

Pinging 172.16.30.0 with 32 bytes of data:

Reply from 172.16.30.1: bytes=32 time<1ms TTL=255
Reply from 172.16.30.1: bytes=32 time<1ms TTL=255
Reply from 172.16.30.1: bytes=32 time<1ms TTL=255
Reply from 172.16.30.1: bytes=32 time<1ms TTL=255

Ping statistics for 172.16.30.0:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

Figure 51: Connectivity Verification using Ping

**RESULT**

OSPF routing was successfully configured on all routers, the routing tables and OSPF neighbor states were displayed, and end-to-end connectivity across the network was verified using `ping`.

# Experiment No. 15

# STANDARD ACCESS CONTROL LIST (ACL)

---

## AIM

You are the network administrator of your college. For the given campus network topology, configure an Access Control List (ACL) to allow only `Host_B` (`172.16.30.2`) to communicate with the `Lab_A` network, and verify the settings by:

(a) Pinging `Host_A` from `Host_B`

(b) Pinging `Host_A` from `Lab_B` and `Lab_C`

## REQUIREMENTS

- Cisco Packet Tracer
- Cisco `1841` and `2621XM` Routers
- Three Cisco `2950-24` Switches
- Three PCs and Two Laptops
- Copper straight-through, cross-over, and Serial DCE/DTE cables

## THEORY

An Access Control List (ACL) is a sequential list of permit or deny statements that filter network traffic based on packet header attributes.

- **Standard ACLs (numbered 1–99):** Filter packets based solely on the **source IPv4 address**. They are placed as close to the destination as possible.
- Every ACL ends with an implicit `deny any` statement.
- Standard ACL syntax:

```
access-list <1-99> {permit | deny} <source-ip> [wildcard-mask]
```

- Applied to an interface using:

```
ip access-group <acl-number> {in | out}
```

## NETWORK TOPOLOGY

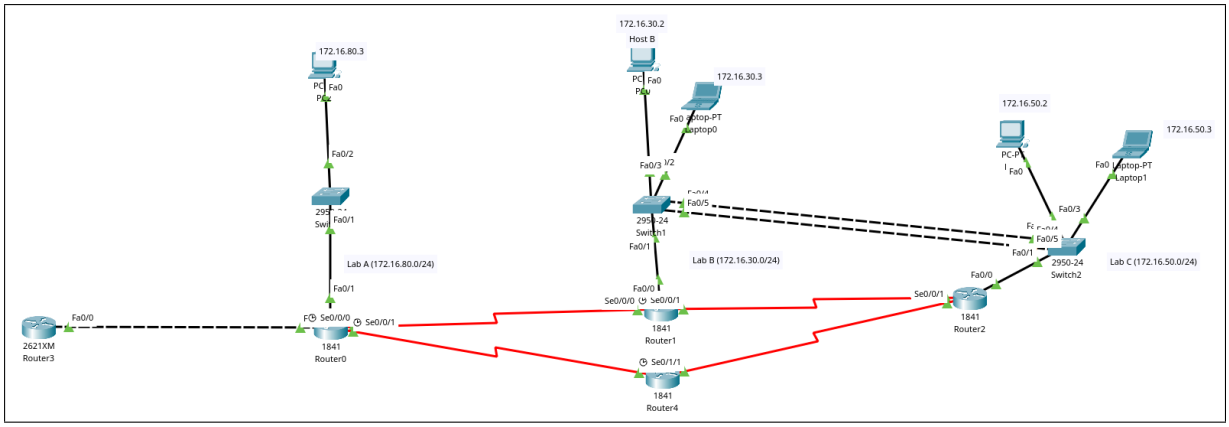



Figure 52: Campus Network Topology with Standard ACL

## IP ADDRESS ALLOCATION

| Device | Interface | IP Address | Subnet Mask | Default Gateway |
|---|---|---|---|---|
| Router3 (2621XM) | FastEthernet0/0 | 172.16.10.1 | 255.255.255.0 | – |
| Router0 (Lab_A) | FastEthernet0/0 | 172.16.10.2 | 255.255.255.0 | – |
| Router0 (Lab_A) | FastEthernet0/1 | 172.16.80.1 | 255.255.255.0 | – |
| Router0 (Lab_A) | Serial0/0/1 (DCE) | 172.16.20.1 | 255.255.255.0 | – |
| Router1 (Lab_B) | FastEthernet0/0 | 172.16.30.1 | 255.255.255.0 | – |
| Router1 (Lab_B) | Serial0/0/0 (DTE) | 172.16.20.2 | 255.255.255.0 | – |
| Router1 (Lab_B) | Serial0/0/1 (DCE) | 172.16.40.1 | 255.255.255.0 | – |
| Router2 (Lab_C) | FastEthernet0/0 | 172.16.50.1 | 255.255.255.0 | – |
| Router2 (Lab_C) | Serial0/0/1 (DTE) | 172.16.40.2 | 255.255.255.0 | – |
| PC2 (Host_A) | FastEthernet0 | 172.16.80.3 | 255.255.255.0 | 172.16.80.1 |
| PC0 (Host_B) | FastEthernet0 | 172.16.30.2 | 255.255.255.0 | 172.16.30.1 |
| Laptop0 (Lab_B) | FastEthernet0 | 172.16.30.3 | 255.255.255.0 | 172.16.30.1 |
| PC1 (Lab_C) | FastEthernet0 | 172.16.50.2 | 255.255.255.0 | 172.16.50.1 |
| Laptop1 (Lab_C) | FastEthernet0 | 172.16.50.3 | 255.255.255.0 | 172.16.50.1 |

Table 11: IP Address Allocation for Campus Network

## PROCEDURE

1. Construct the campus network topology in Cisco Packet Tracer using routers, switches, PCs, and laptops as shown in the topology.

2. Configure IP addresses on all router interfaces, PCs, and laptops according to the IP Address Allocation table.

3. Configure dynamic routing (RIPv2) on all routers so that all subnets can communicate initially.
4. Create a Standard ACL (ACL `10`) on `Router0` (`Lab_A`) that permits only `Host_B` (`172.16.30.2`) and denies all other sources.
5. Bind ACL `10` outbound (`out`) on the LAN interface (`FastEthernet0/1` / `FastEthernet0/0`) of `Router0` (`Lab_A`).
6. Test connectivity by pinging `Host_A` (`172.16.80.3`) from `Host_B` (`172.16.30.2`), and verify that pings from other devices in `Lab_B` (`Laptop0` / `Router1`) and `Lab_C` (`PC1` / `Laptop1` / `Router2`) are blocked.

## CONFIGURATION

### Router Interface & RIPv2 Configuration

### Router0 (Lab_A)

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.10.2 255.255.255.0
no shutdown
exit
interface fastethernet0/1
ip address 172.16.80.1 255.255.255.0
no shutdown
exit
interface serial0/0/1
ip address 172.16.20.1 255.255.255.0
clock rate 64000
no shutdown
exit
router rip
version 2
no auto-summary
network 172.16.10.0
network 172.16.20.0
network 172.16.80.0
exit
```

### Router1 (Lab_B)

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.30.1 255.255.255.0
no shutdown
exit
interface serial0/0/0
ip address 172.16.20.2 255.255.255.0
no shutdown
exit
interface serial0/0/1
ip address 172.16.40.1 255.255.255.0
clock rate 64000
no shutdown
exit
router rip
version 2
no auto-summary
network 172.16.20.0
network 172.16.30.0
network 172.16.40.0
end
```

**Router2 (Lab_C)**

```
enable
configure terminal
interface fastethernet0/0
ip address 172.16.50.1 255.255.255.0
no shutdown
exit
interface serial0/0/1
ip address 172.16.40.2 255.255.255.0
no shutdown
exit
router rip
version 2
no auto-summary
network 172.16.40.0
network 172.16.50.0
end
```

### Standard ACL Configuration on Router0 (Lab_A)

```
enable
configure terminal
access-list 10 permit host 172.16.30.2
access-list 10 deny any
interface fastethernet0/1
ip access-group 10 out
exit
interface fastethernet0/0
ip access-group 10 out
end
```

## VERIFICATION

(a) **Pinging** Host_A (172.16.80.3) **from** Host_B (172.16.30.2)**:**

```
C:\> ping 172.16.80.3
! Result: Replies received successfully (Permitted by
ACL 10).
```

(b) **Pinging** Host_A (172.16.80.3) **from** Lab_B **and** Lab_C**:**

```
! From Lab_B (Laptop0 172.16.30.3 or Router1):
C:\> ping 172.16.80.3
! Result: Destination host unreachable (Blocked by ACL
10)

! From Lab_C (PC1 172.16.50.2 / Laptop1 172.16.50.3 or
Router2):
C:\> ping 172.16.80.3
! Result: Destination host unreachable (Blocked by ACL
10)
```

**OUTPUT**

```
C:\>ping 172.16.80.3

Pinging 172.16.80.3 with 32 bytes of data:

Reply from 172.16.80.3: bytes=32 time=27ms TTL=126
Reply from 172.16.80.3: bytes=32 time=35ms TTL=126
Reply from 172.16.80.3: bytes=32 time=26ms TTL=126
Reply from 172.16.80.3: bytes=32 time=29ms TTL=126

Ping statistics for 172.16.80.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 26ms, Maximum = 35ms, Average = 29ms

C:\>
```

Figure 53: Successful Ping from Host_B to Host_A

```
Router>enable
Router#ping 172.16.80.3

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.80.3, timeout is 2 seconds:
UUUUU
Success rate is 0 percent (0/5)

Router#
```

Figure 54: Blocked Ping from Lab_B to Host_A

```
Router>enable
Router#ping 172.16.80.3

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.80.3, timeout is 2 seconds:
UUUUU
Success rate is 0 percent (0/5)

Router#
```

Figure 55: Blocked Ping from Lab_C to Host_A

**RESULT**

Standard ACL was successfully configured on `Lab_A` (`Router0`) to permit only `Host_B` (`172.16.30.2`) to communicate with `Lab_A`, and verified by pinging `Host_A` (`172.16.80.3`) from `Host_B`, `Lab_B`, and `Lab_C`.

# Experiment No. 16

# SUBNETTING AND EXTENDED ACL CONFIGURATION

---

## AIM

You are the network administrator of your college assigned the network address `140.80.0.0` with 20 subnets. The Central Computing Facility (CCF) resides in the 4th subnet, and the CSE Hackathon website is hosted on a server assigned the 7th address in the 16th subnet. Configure an Extended ACL to block students in CCF from accessing the hackathon website while allowing all other services on the server.

## REQUIREMENTS

- Cisco Packet Tracer
- One Cisco `1841` Router
- Two Cisco `2950-24` Switches
- Three Laptops
- One Server
- Copper straight-through cables

## THEORY AND SUBNET DESIGN

### 1. Subnetting Calculation

- **Given Network Address:** `140.80.0.0` (Class B, Default Subnet Mask: `255.255.0.0` or `/16`)
- **Required Number of Subnets:** 20
- **Number of Host Bits Borrowed ($n$):** Since $2^5 = 32 \geq 20$, we borrow $n = 5$ bits from the third octet.
- **Custom Subnet Mask:** `/21` $\rightarrow$ `255.255.248.0` (Wildcard Mask: `0.0.7.255`)
- **Block Size in 3rd Octet:** $2^{8-5} = 2^3 = 8$

### 2. Subnet Address Identification

Using a block size of 8 in the third octet (where the $k$th subnet starts at $k \times 8$):

- **4th Subnet (CCF Subnet):** $4 \times 8 = 32 \rightarrow$ `140.80.32.0/21`
- **16th Subnet (Admin / CSE Network):** $16 \times 8 = 128 \rightarrow$ `140.80.128.0/21`
- **Hackathon Server (7th usable address in 16th Subnet):** `140.80.128.7`

| Subnet | Network Address | Usable Host IP Range | Broadcast Address |
|---|---|---|---|
| 1st Subnet | 140.80.8.0/21 | 140.80.8.1 – 140.80.15.254 | 140.80.15.255 |
| 2nd Subnet | 140.80.16.0/21 | 140.80.16.1 – 140.80.23.254 | 140.80.23.255 |
| 3rd Subnet | 140.80.24.0/21 | 140.80.24.1 – 140.80.31.254 | 140.80.31.255 |
| **4th (CCF)** | **140.80.32.0/21** | **140.80.32.1 – 140.80.39.254** | **140.80.39.255** |
| **16th (CSE)** | **140.80.128.0/21** | **140.80.128.1 – 140.80.135.254** | **140.80.135.255** |

Table 12: Subnet Allocation Table

### 3. Extended ACL Theory

Extended ACLs (numbered **100–199**) filter traffic based on source IP, destination IP, protocol (`tcp`, `udp`, `icmp`, `ip`), and port number (e.g., port `80` / `www` for HTTP). Because they specify both source and destination addresses, they are placed as close to the source (`CCF Subnet`) as possible.

## NETWORK TOPOLOGY

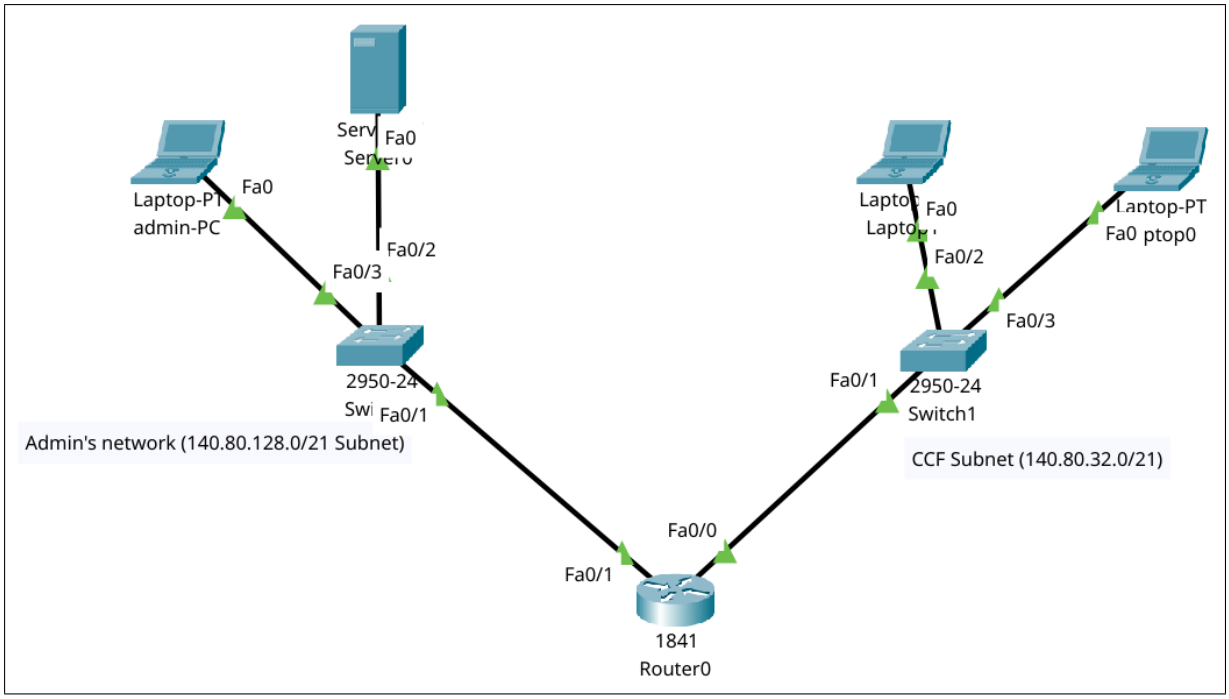



Figure 56: College Network Topology for CCF and CSE Hackathon Server

## IP ADDRESS ALLOCATION

| Device | Interface | IP Address | Subnet Mask | Default Gateway |
|---|---|---|---|---|
| Router0 | FastEthernet0/0 | 140.80.32.1 | 255.255.248.0 | – |
| Router0 | FastEthernet0/1 | 140.80.128.1 | 255.255.248.0 | – |
| Laptop0 | FastEthernet0 | 140.80.32.2 | 255.255.248.0 | 140.80.32.1 |
| Laptop1 | FastEthernet0 | 140.80.32.3 | 255.255.248.0 | 140.80.32.1 |
| Server0 | FastEthernet0 | 140.80.128.7 | 255.255.248.0 | 140.80.128.1 |
| admin-PC | FastEthernet0 | 140.80.128.2 | 255.255.248.0 | 140.80.128.1 |

Table 13: IP Address Allocation

## CONNECTIVITY

| Device | Interface | Connected Device | Interface |
|---|---|---|---|
| Router0 | FastEthernet0/0 | Switch1 | FastEthernet0/1 |
| Switch1 | FastEthernet0/2 | Laptop1 | FastEthernet0 |
| Switch1 | FastEthernet0/3 | Laptop0 | FastEthernet0 |
| Router0 | FastEthernet0/1 | Switch0 | FastEthernet0/1 |
| Switch0 | FastEthernet0/2 | Server0 | FastEthernet0 |
| Switch0 | FastEthernet0/3 | admin-PC | FastEthernet0 |

Table 14: Network Connectivity

## PROCEDURE

1. Calculate the subnet mask (`255.255.248.0`), the 4th subnet address for CCF (`140.80.32.0/21`), and the 7th host IP in the 16th subnet for the Hackathon Server (`140.80.128.7`).
2. Connect `Laptop0` and `Laptop1` (CCF) to `Router0` (`Fa0/0`) via `Switch1`, and connect `Server0` and `admin-PC` to `Router0` (`Fa0/1`) via `Switch0`.
3. Configure the router interfaces, laptops, and `Server0` with their respective IP addresses, `/21` subnet masks, and default gateways.
4. Ensure HTTP and other services (FTP/ICMP) are enabled on `Server0`.
5. Configure an Extended ACL (ACL `100`) on `Router0` to deny TCP port `80` (`www`) and port `443` traffic from the CCF subnet (`140.80.32.0 0.0.7.255`) to `Server0` (`host 140.80.128.7`), and permit all other IP traffic.
6. Apply ACL `100` inbound (`in`) on interface `FastEthernet0/0` of `Router0`.
7. Verify that accessing `http://140.80.128.7` from `Laptop0` or `Laptop1` is blocked (`Request Timeout`), while `ping 140.80.128.7` and web access from `admin-PC` succeed.

## CONFIGURATION

### Router0 Configuration

```
enable
configure terminal
```

```
! Interface connected to 4th Subnet (CCF Subnet)
interface fastethernet0/0
ip address 140.80.32.1 255.255.248.0
no shutdown
exit

! Interface connected to 16th Subnet (Admin's Network / CSE
Server)
interface fastethernet0/1
ip address 140.80.128.1 255.255.248.0
no shutdown
exit

! Extended ACL 100 to block only HTTP/HTTPS access from CCF
to Server0
access-list 100 deny tcp 140.80.32.0 0.0.7.255 host
140.80.128.7 eq www
access-list 100 deny tcp 140.80.32.0 0.0.7.255 host
140.80.128.7 eq 443
access-list 100 permit ip any any

! Apply ACL inbound on CCF interface (Fa0/0)
interface fastethernet0/0
ip access-group 100 in
end
```

## VERIFICATION

1. **Web Browser Check from CCF** (`Laptop0` / `Laptop1`)**:** Navigate to `http://140.80.128.7`. The request fails with `Request Timeout`, confirming website access from CCF is blocked.

2. **Other Services Check (ICMP / FTP) from CCF** (`Laptop0` / `Laptop1`)**:**

```
C:\> ping 140.80.128.7
C:\> ftp 140.80.128.7
```

   Both ICMP echo replies and FTP connection succeed, confirming other services on `Server0` remain accessible.

## OUTPUT

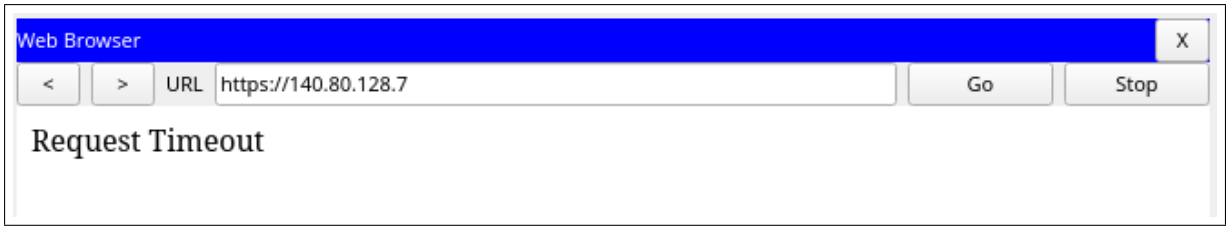



Figure 57: Blocked Website Access (HTTP Request Timeout) from CCF Laptop

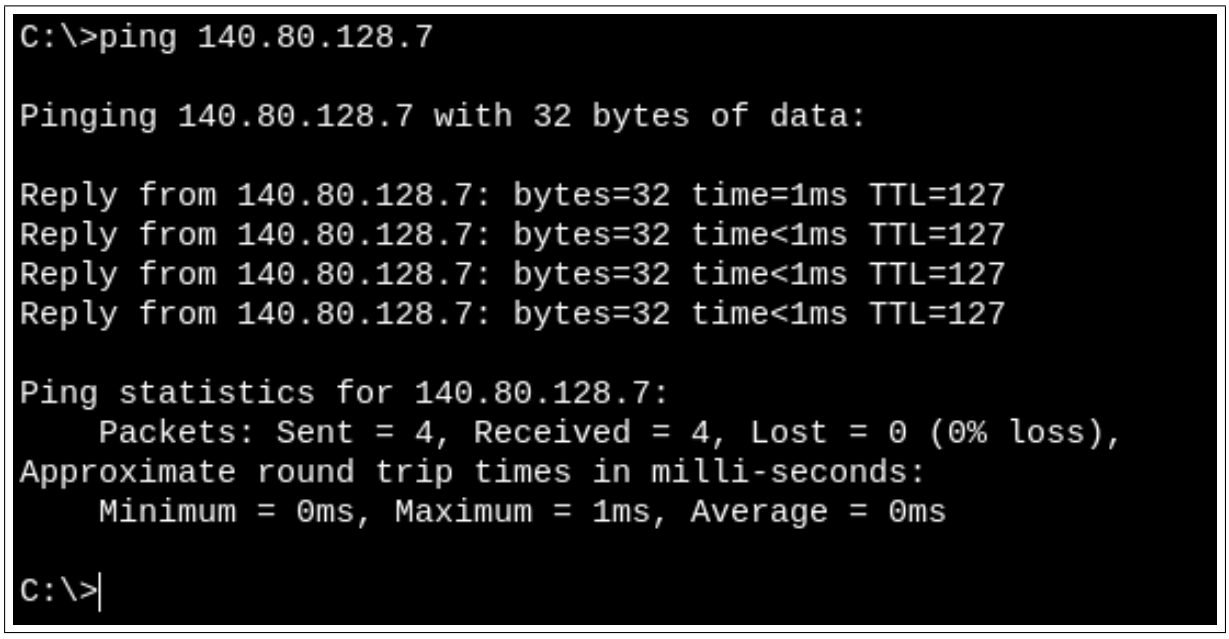



Figure 58: Successful Ping and Non-HTTP Access from CCF Laptop to Server0

## RESULT

The college network `140.80.0.0/16` was subnetted into `/21` subnets, and an Extended ACL was successfully configured on `Router0` to block HTTP website access from the CCF subnet (`140.80.32.0/21`) to the Hackathon Server (`140.80.128.7`) while allowing all other network services.